

Kolmogorov-Arnold Networks Meet Science

Ziming Liu^{✉*} and Max Tegmark[✉]*Massachusetts Institute of Technology and The NSF Institute for Artificial Intelligence and Fundamental Interactions, Massachusetts, USA*Pingchuan Ma[✉] and Wojciech Matusik[✉]*Massachusetts Institute of Technology and Computer Science and Artificial Intelligence Laboratory (CSAIL), Massachusetts, USA*Yixuan Wang[✉]*California Institute of Technology, California, USA*

(Received 1 December 2024; accepted 30 September 2025; published 17 December 2025)

A major challenge of AI plus science lies in its inherent incompatibility: Today's AI is primarily based on connectionism, while science depends on symbolism. To bridge the two worlds, we propose a framework to seamlessly synergize Kolmogorov-Arnold networks (KANs) and science. The framework highlights KANs' usage for three aspects of scientific discovery: identifying relevant features, revealing modular structures, and discovering symbolic formulas. The synergy is bidirectional: science to KAN (incorporating scientific knowledge into KANs), and KAN to science (extracting scientific insights from KANs). We highlight major new functionalities in PYKAN: (1) MultKAN, KANs with multiplication nodes, (2) kanpiler, a KAN compiler that compiles symbolic formulas into KANs; (3) tree converter, convert KANs (or any neural networks) into tree graphs. Based on these tools, we demonstrate KANs' capability to discover various types of physical laws, including conserved quantities, Lagrangians, symmetries, and constitutive laws.

DOI: [10.1103/4t7t-v19l](https://doi.org/10.1103/4t7t-v19l)Subject Areas: Computational Physics,
Interdisciplinary Physics, Mechanics

I. INTRODUCTION

In recent years, AI plus science has emerged as a promising new field, leading to significant scientific advancements including predicting protein folding [1], proving automated theorems [2,3], and weather forecasting [4], among others. A common thread among these tasks is that they can all be well formulated into problems with clear objectives, optimizable by black-box AI systems. While this paradigm works exceptionally well for application-driven science, a different kind of science exists: curiosity-driven science. In curiosity-driven research, the procedure is more exploratory, often lacking clear goals beyond “gaining more understanding.” To clarify, curiosity-driven science is far from useless—quite the opposite. The scientific knowledge and understanding gained through

curiosity often lay a solid foundation for tomorrow's technology and foster a wide range of applications.

Although both application-driven and curiosity-driven science are invaluable and irreplaceable, they ask different questions. When astronomers observe the motion of celestial bodies, application-driven researchers focus on predicting their future states, while curiosity-driven researchers explore the physics behind the motion. Another example is AlphaFold, which, despite its tremendous success in predicting protein structures, remains in the realm of application-driven science because it does not provide new knowledge at a more fundamental level (e.g., atomic forces). Hypothetically, AlphaFold must have uncovered important unknown physics to achieve its highly accurate predictions. However, this information remains hidden from us, leaving AlphaFold largely a black box. Therefore, we advocate for new AI paradigms to support curiosity-driven science. This new paradigm of AI plus science demands a higher degree of interpretability and interactivity in AI tools so that they can be seamlessly integrated into scientific research.

Recently, a new type of neural network called the Kolmogorov-Arnold network (KAN) [5] has shown promise for science-related tasks. Unlike multilayer

*Contact author: zmliu@mit.edu

Published by the American Physical Society under the terms of the [Creative Commons Attribution 4.0 International](https://creativecommons.org/licenses/by/4.0/) license. Further distribution of this work must maintain attribution to the author(s) and the published article's title, journal citation, and DOI.

perceptrons (MLPs), which have fixed activation functions on nodes, KANs feature learnable activation functions on edges. KANs admit a better parametrization than MLPs in the sense that they can represent MLPs with the same number of parameters, whereas MLPs need a lot more parameters to represent KANs; at the same time, KANs can learn the high-frequency components efficiently during the optimization process, while MLPs suffer, both established in a recent work [6]. KANs admit a better parametrization than MLPs and can learn the high-frequency components better [6]. Because KANs can decompose high-dimensional functions into one-dimensional functions, interpretability can be gained by symbolically regressing these 1D functions. However, their definition of interpretability is somewhat narrow, equating it almost exclusively with the ability to extract symbolic formulas. This limited definition restricts their scope, as symbolic formulas are not always necessary or feasible in science. For example, while symbolic equations are powerful and prevalent in physics, systems in chemistry and biology are often too complex to be represented by such equations. In these fields, modular structures and key features may be sufficient to characterize interesting aspects of these systems. Another overlooked aspect is the reverse task of embedding knowledge into KANs: How can we incorporate prior knowledge into KANs, in the spirit of physics-informed learning?

We enhance and extend KANs to make them easily used for curiosity-driven science. The goal of this paper is to synergize Kolmogorov-Arnold networks and science. On the one hand, we explore ways to build scientific knowledge into KANs (Sec. III). On the other hand, we propose techniques to extract scientific knowledge from KANs (Sec. IV). Concrete examples are provided in Sec. V.

To be more concrete, scientific explanations may have different levels ranging from the coarsest, easiest, and correlational to the finest, hardest, and causal:

- (i) Important features: For example, “ y is fully determined by x_1 and x_2 , while other factors do not matter.” In other words, there exists a function f such that $y = f(x_1, x_2)$.
- (ii) Modular structures: For instance, “ x_1 and x_2 contribute to y independently in an additive way.” This means there exist functions g and h such that $y = g(x_1) + h(x_2)$.
- (iii) Symbolic formulas: For example, “ y depends on x_1 as a sine function and on x_2 as an exponential function.” In other words, $y = \sin(x_1) + \exp(x_2)$.

This paper reports on how to incorporate and extract these properties from KANs. The structure of the paper is as follows (illustrated in Fig. 1): In Sec. II, we augment the original KAN with multiplication nodes, introducing a new model called MultKAN. In Sec. III, we explore ways to embed scientific inductive biases into KANs, focusing on important features (Sec. III A), modular structures (Sec. III B), and symbolic formulas (Sec. III C). In Sec. IV, we propose methods to extract scientific

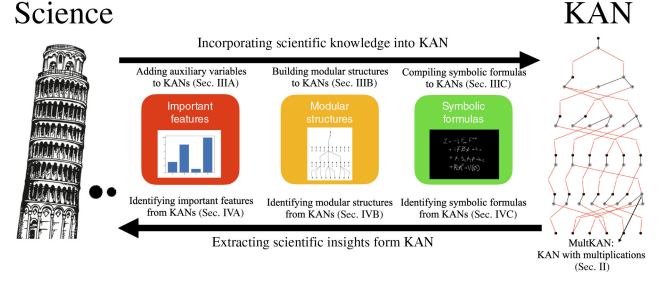


FIG. 1. Synergizing science and the KAN.

knowledge from KANs, again covering important features (Sec. IV A), modular structures (Sec. IV B), and symbolic formulas (Sec. IV C). In Sec. V, we apply KANs to various scientific discovery tasks using the tools developed in the previous sections. These tasks include discovering conserved quantities, symmetries, Lagrangians, and constitutive laws. The codes are available [7] and can also be installed via `PIP INSTALL PYKAN`. Although the title of the paper is “KAN2.0,” the release version of PYKAN is 0.2.x.

II. MULTKAN: AUGMENTING KANs WITH MULTIPLICATIONS

The Kolmogorov-Arnold representation theorem (KART) states that any continuous high-dimensional function can be decomposed into a finite composition of univariate continuous functions and additions:

$$f(\mathbf{x}) = f(x_1, \dots, x_n) = \sum_{q=1}^{2n+1} \Phi_q \left(\sum_{p=1}^n \phi_{q,p}(x_p) \right). \quad (1)$$

This implies that addition is the only true multivariate operation, while other multivariate operations (including multiplication) can be expressed as additions combined with univariate functions. For example, to multiply two positive numbers x and y , we can express this as $xy = \exp(\log x + \log y)$ [8], whose right-hand side consists only of addition and univariate functions ($\log$ and $\exp$). However, given the prevalence of multiplications in both science and everyday life, it is desirable to explicitly include multiplications in KANs, which could potentially enhance both interpretability and capacity.

A. Kolmogorov-Arnold networks

While the KART Eq. (1) corresponds to a two-layer network, Liu *et al.* [5] managed to extend it to arbitrary depths by recognizing that seemingly different outer functions Φ_q and inner functions $\phi_{q,p}$ can be unified through their proposed KAN layers. A depth- L KAN can be constructed simply by stacking L KAN layers. The shape of a depth- L KAN is represented by an integer array $[n_0, n_1, \dots, n_L]$ where n_l denotes the number of neurons in the l th neuron layers. The l th KAN layer, with

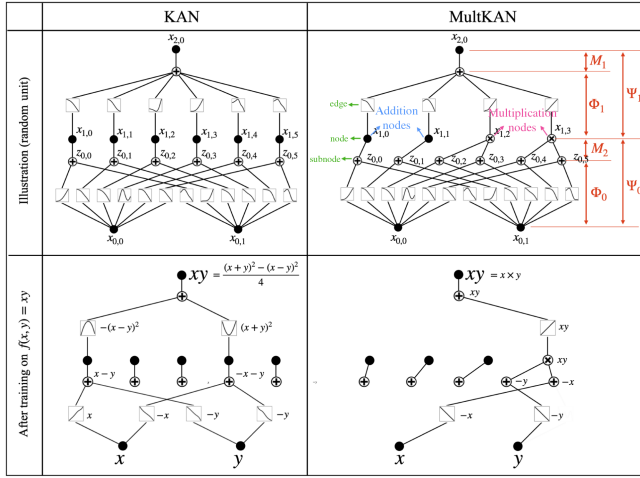


FIG. 2. Top: comparing KAN and MultKAN diagrams. MultKAN has extra multiplication layers $\mathbf{M}$. Bottom: after training on $f(x, y) = xy$, KAN learns an algorithm requiring two addition nodes, while MultKAN requires only one multiplication node.

n_l input dimensions and n_{l+1} output dimensions, transforms an input vector $\mathbf{x}_l \in \mathbb{R}^{n_l}$ to $\mathbf{x}_{l+1} \in \mathbb{R}^{n_{l+1}}$,

$$\mathbf{x}_{l+1} = \underbrace{\begin{pmatrix} \phi_{l,1,1}(\cdot) & \phi_{l,2,1}(\cdot) & \cdots & \phi_{l,n_l,1}(\cdot) \\ \phi_{l,1,2}(\cdot) & \phi_{l,2,2}(\cdot) & \cdots & \phi_{l,n_l,2}(\cdot) \\ \vdots & \vdots & \ddots & \vdots \\ \phi_{l,1,n_{l+1}}(\cdot) & \phi_{l,2,n_{l+1}}(\cdot) & \cdots & \phi_{l,n_l,n_{l+1}}(\cdot) \end{pmatrix}}_{\Phi_l} \mathbf{x}_l, \quad (2)$$

and the whole network is a composition of L KAN layers, i.e.,

$$\text{KAN}(\mathbf{x}) = (\Phi_{L-1} \circ \cdots \circ \Phi_1 \circ \Phi_0) \mathbf{x}. \quad (3)$$

In diagrams, KANs can be intuitively visualized as a network consisting of nodes (summation) and edges (learnable activations), as shown in Fig. 2, top left. When trained on the dataset generated from $f(x, y) = xy$, the KAN (Fig. 2, bottom left) uses two addition nodes, making it unclear what the network is doing. However, after some consideration, we realize it leverages the equality $xy = [(x+y)^2 - (x-y)^2]/4$ but this is far from obvious.

B. Multiplicative Kolmogorov-Arnold networks

To explicitly introduce multiplication operations, we propose the MultKAN, which can reveal multiplicative structures in data more clearly. A MultKAN (shown in Fig. 2, top right) is similar to a KAN, with both having standard KAN layers. We refer to the input nodes of a KAN layer as nodes, and the output nodes of a KAN layer

subnodes. The difference between KAN and MultKAN lies in the transformations from the current layer's subnodes to the next layer's nodes. In KANs, nodes are directly copied from the previous layer's subnodes. In MultKANs, some nodes (addition nodes) are copied from corresponding subnodes, while other nodes (multiplication nodes) perform multiplication on k subnodes from the previous layer. For simplicity, we set $k = 2$ below [9].

Based on the MultKAN diagram (Fig. 2, top right), it can be intuitively understood that a MultKAN is a normal KAN with optional multiplications inserted in. To be mathematically precise, we define the following notations: The number of addition (multiplication) operations in layer l are denoted as n_l^a (n_l^m), respectively. These are collected into arrays: addition width $\mathbf{n}^a \equiv [n_0^a, n_1^a, \dots, n_L^a]$ and multiplication width $\mathbf{n}^m \equiv [n_0^m, n_1^m, \dots, n_L^m]$. When $n_0^m = n_1^m = \cdots = n_L^m = 0$, the MultKAN reduces to a KAN. For example, Fig. 2 (top right) shows a MultKAN with $\mathbf{n}^a = [2, 2, 1]$ and $\mathbf{n}^m = [0, 2, 0]$.

A MultKAN layer consists of a standard KANLayer Φ_l and a multiplication layer $\mathbf{M}_l$. Φ_l takes in an input vector $\mathbf{x}_l \in \mathbb{R}^{n_l^a + n_l^m}$ and outputs $\mathbf{z}_l = \Phi_l(\mathbf{x}) \in \mathbb{R}^{n_{l+1}^a + 2n_{l+1}^m}$. The multiplication layer consists of two parts: The multiplication part performs multiplications on subnode pairs, while the other part performs identity transformation. Written in PYTHON, $\mathbf{M}_l$ transforms $\mathbf{z}_l$ as follows:

$$\mathbf{M}_l(\mathbf{z}_l) = \text{concat}(\mathbf{z}_l[:n_{l+1}^a], \mathbf{z}_l[n_{l+1}^a : : 2] \odot \mathbf{z}_l[n_{l+1}^a + 1 : : 2]),$$

where $\odot$ is elementwise multiplication. The MultKANLayer can be succinctly represented as $\Psi_l \equiv \mathbf{M}_l \circ \Phi_l$. The whole MultKAN is thus

$$\text{MultKAN}(\mathbf{x}) = (\Psi_L \circ \Psi_{L-1} \circ \cdots \circ \Psi_1 \circ \Psi_0) \mathbf{x}. \quad (4)$$

Since there are no trainable parameters in multiplication layers, all sparse regularization techniques (e.g., ℓ_1 and entropy regularization) for KANs [5] can be directly applied to MultKANs. For the multiplication task $f(x, y) = xy$, the MultKAN indeed learns to use one multiplication node, making it perform simple multiplication, as all the learned activation functions are linear (Fig. 2, bottom right).

We recall the approximation theories established for KANs in Refs. [5,6]. In particular, they also apply to MultKANs, as stated in the theorem below.

Theorem 1. Let $\mathbf{x} = (x_1, x_2, \dots, x_n)$. Suppose that a function $f(\mathbf{x})$ admits a representation

$$f = (\Phi_{L-1} \circ \Phi_{L-2} \circ \cdots \circ \Phi_1 \circ \Phi_0) \mathbf{x}, \quad (5)$$

where each one of the $\Phi_{l,i,j}$ is $(k+1)$ -times continuously differentiable. Then there exists a constant C depending on f and its representation, such that we have the following approximation bound in terms of the grid size G :

There exist k th-order B -spline functions $\Phi_{l,i,j}^G$ such that for any $0 \leq m \leq k$, we have the bound for the C^m norm

$$\|f - (\Phi_{L-1}^G \circ \Phi_{L-2}^G \circ \dots \circ \Phi_1^G \circ \Phi_0^G)\mathbf{x}\|_{C^m} \leq CG^{-k-1+m}. \quad (6)$$

Theorem 2. Let $\Omega \subset \mathbb{R}^d$ be a bounded domain with smooth boundary $s > 0$, and $1 \leq p, q \leq \infty$ be such that $1/q - 1/p < s/d$. This guarantees that the compact Sobolev embedding $W^s(L_q(\Omega)) \subset \subset L_p(\Omega)$ holds. Let $W_0 := W_0(d)$ be a fixed width (depending upon the input dimension d). Then, for any $f \in W^s(L_q(\Omega))$ and any $L \geq 1$, there exists a MultKAN g with width W_0 , depth L , and grid size $G = 2$ with k th-order B -spline functions such that

$$\|f - g\|_{L_p(\Omega)} \leq CL^{-2s/d}, \quad (7)$$

where C is a constant independent of L .

Although KANs have previously been seen as a special case of MultKANs, we extend the definition and treat “KAN” and “MultKAN” as synonyms. By default, when we refer to KANs, multiplication is allowed. If we specifically refer to a KAN without multiplication, we will explicitly state so.

$$\text{model.augment_input}(\text{ori_var}, \text{aux_var}, \text{dataset}). \quad (8)$$

As an example, consider the formula for relativistic mass $m(m_0, v, c) = m_0 / \sqrt{1 - (v/c)^2}$ where m_0 is the rest mass, v is the velocity of the point mass, and c is the speed of light. Since physicists often work with dimensionless numbers $\beta \equiv v/c$ and $\gamma \equiv 1/\sqrt{1 - \beta^2} \equiv 1/\sqrt{1 - (v/c)^2}$, they might introduce β and γ alongside v and c as inputs. Figure 3 shows KANs trained as a regression problem with and without these auxiliary variables: Figure 3(a) illustrates the KAN compiled from the symbolic formula (see Sec. III C for the KAN compiler), which requires five edges; Figs. 3(b) and 3(c) show KANs with auxiliary variables requiring only two or three edges and achieving losses of 10^{-6} and 10^{-4} , respectively. Note that Figs. 3(b) and 3(c) differ only in random seeds. Seed 1 represents a suboptimal solution because it also identifies $\beta = v/c$ as a key feature. However, they both give the same output $1/\sqrt{1 - (v/c)^2}$, since in Fig. 3(c), $1/\sqrt{1 - (v/c)^2}$ is just represented as a sum of $1/\sqrt{1 - (v/c)^2}$ and a non-linear function of v/c in the KAN formulation. This is not surprising, as in the classical limit $v \ll c$, $\gamma \equiv 1/\sqrt{1 - (v/c)^2} \approx 1 + (v/c)^2/2 = 1 + \beta^2/2$. The variation due to different seeds can be seen either as a feature or a bug: As a feature, this diversity can help find suboptimal

III. SCIENCE TO KANs

In science, domain knowledge is crucial, allowing us to work effectively even with small or zero data. Therefore, it is beneficial to adopt a physics-informed approach for KANs: We should incorporate available inductive biases into KANs while preserving their flexibility to discover new physics from the data.

We explore three types of inductive biases that can be integrated into KANs. From the coarsest, easiest, and correlational to the finest, hardest, and causal, they are important features (Sec. III A), modular structures (Sec. III B), and symbolic formulas (Sec. III C).

A. Adding important features to KANs

In a regression problem, the goal is to find a function f such that $y = f(x_1, x_2, \dots, x_n)$. Suppose we want to introduce an auxiliary input variable $a = a(x_1, x_2, \dots, x_n)$, transforming the function to $y = f(x_1, \dots, x_n, a)$. Although the auxiliary variable a does not add new information, it can increase the expressive power of the neural network. This is because the network does not need to expend resources to calculate the auxiliary variable. Additionally, the computations may become simpler, leading to improved interpretability. Users can add auxiliary features to inputs using the `augment_input` method:

solutions, which may nevertheless offer interesting insights; as a bug, it can be eliminated using the hypothesis testing method proposed in Sec. IV C.

B. Building modular structures into KANs

Modularity is prevalent in nature: For example, the human cerebral cortex is divided into several functionally distinct modules, each of these modules responsible for

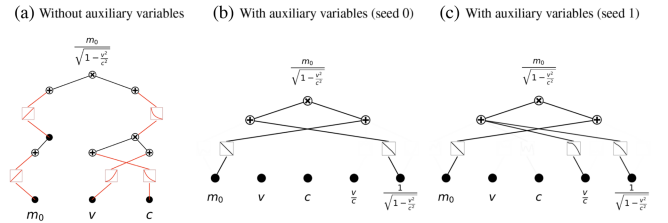


FIG. 3. Adding auxiliary variables to inputs enhances interpretability. For the relativistic mass equation $m = m_0 / \sqrt{1 - v^2/c^2}$, (a) a two-layer KAN is needed if only (m_0, v, c) are used as inputs. (b) If we add $\beta \equiv v/c$ and $\gamma \equiv 1/\sqrt{1 - \beta^2}$ as auxiliary variables to KANs, a one-layer KAN suffices (seed 0). (c) Seed 1 finds a different solution, which is suboptimal and can be avoided through hypothesis testing (Sec. IV C).

specific tasks such as perception or decision-making. This modularity simplifies the understanding of neural networks, as it allows us to interpret clusters of neurons collectively rather than analyzing each neuron individually. Structural modularity is characterized by clusters of connections where intracluster connections are much stronger than intercluster ones. To enforce modularity, we introduce the module method, which preserves intracluster connections while removing intercluster connections. The modules are specified by users. The syntax is

```
model.module(start_layer_id,
'nodes_id]->[subnodes_id]->[nodes_id]...').
(9)
```

For example, if a user wants to assign specific nodes or subnodes to a module, say, the zeroth node in layer 1, the first and third subnodes in layer 1, the first and third nodes in layer 2, they might use `module(1, '[0]->[1,3]->[1,3]')`. To be concrete, there are two types of modularity: separability and symmetry.

1. Separability

We say a function is considered separable if it can be expressed as a sum or product of functions of nonoverlapping variable groups. For example, a four-variable function $f(x_1, x_2, x_3, x_4)$ is maximally multiplicatively separable if it has the form $f_1(x_1)f_2(x_2)f_3(x_3)f_4(x_4)$, creating four distinct groups (1), (2), (3), (4). Users can create these modules by calling the module method four times: `module(0, '[i]->[i]')`, $i = 0, 1, 2, 3$ shown in Fig. 4(a). The final call may be skipped since the first three are sufficient to define the groups. Weaker forms of multiplicative separability might be $f_1(x_1, x_2)f_2(x_3, x_4)$ [calling `module(0, '[0,1]->[0,1]')`] or $f_1(x_1) \times f_2(x_2, x_3, x_4)$ [calling `module(0, '[0]->[0]')`].

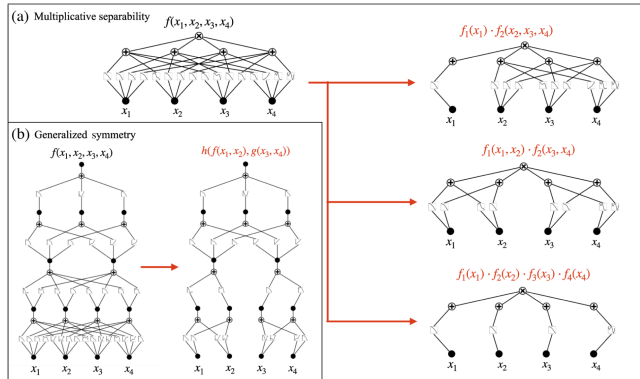


FIG. 4. Building modular structures for KANs. (a) Multiplicative separability, (b) symmetries.

2. Generalized symmetry

We say a function is symmetric in variables (x_1, x_2) if $f(x_1, x_2, x_3, \dots) = g(h(x_1, x_2), x_3, \dots)$. This property is termed symmetry because the value of f remains unchanged as long as $h(x_1, x_2)$ is constant, even if x_1 and x_2 vary. For example, a function f is rotational invariant in 2D if $f(x_1, x_2) = g(r)$, where $r \equiv \sqrt{x_1^2 + x_2^2}$. When symmetry involves only a subset of variables, it can be considered hierarchical since x_1 and x_2 interact first through h (two-layer KAN), and then h interacts with other variables via g (two-layer KAN). Suppose a four-variable function has a hierarchical form $f(x_1, x_2, x_3, x_4) = h(f(x_1, x_2), g(x_3, x_4))$, as illustrated in Fig. 4(b). We can use the module method to create this structure by calling `module(0, '[0,1]->[0,1]->[0,1]->[0]')`, ensuring that the variable groups (x_1, x_2) and (x_3, x_4) do not interact in the first two layers.

C. Compiling symbolic formulas into KANs

Scientists often find satisfaction in representing complex phenomena through symbolic equations. However, while these equations are concise, they may lack the expressive

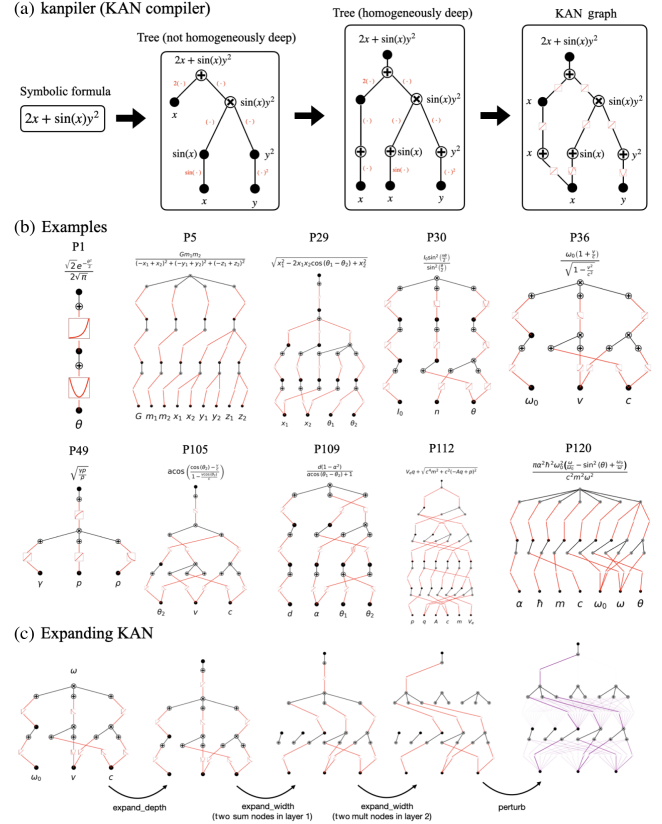


FIG. 5. KAN compiler (kanpiler) converts symbolic expressions to KANs. (a) How kanpiler works: the symbolic formula is first parsed to an expression tree, which is then converted to a KAN. (b) Applying KANs to ten equations (selected from the Feynman dataset). (c) Expand a compiled KAN to increase its expressive power.

power needed to capture all nuances due to their specific functional forms. In contrast, neural networks are highly expressive but may inefficiently spend training time and data to learn domain knowledge already known to scientists. To leverage the strengths of both approaches, we propose a two-step procedure: (1) compile symbolic equations into KANs and (2) fine-tune these KANs using data. The first step aims to embed known domain knowledge into KANs, while the second step focuses on learning new “physics” from the data.

1. *kanpiler*

The goal of the *kanpiler* is to convert a symbolic formula into a KAN. The process, illustrated in Fig. 5(a), involves three main steps: (1) The symbolic formula is parsed into a

tree structure, where nodes represent expressions, and edges denote operations and functions. (2) This tree is then modified to align with the structure of a KAN graph. Modifications include moving all leaf nodes to the input layer via dummy edges, and adding dummy subnodes or nodes to match the KAN architecture. These dummy edges, nodes, and subnodes perform only identity transformation. (3) The variables are combined in the first layer, effectively converting the tree into a graph. For visual clarity, 1D curves are placed on edges to represent functions. We have benchmarked the *kanpiler* on the Feynman dataset, and it successfully handles all 120 equations. Examples are shown in Fig. 5(b). The *kanpiler* takes input variables (as SYMPY symbols) and output expression (as a SYMPY expression), and returns a KAN model

$$\text{model} = \text{kanpiler}(\text{input_variables}, \text{output_expression}). \quad (10)$$

Note that the returned KAN model is in the symbolic mode; i.e., the symbolic functions are exactly encoded. If we instead use cubic splines to approximate these symbolic functions, we get MSE losses $\ell \propto N^{-8}$ [5] based on the approximation order for 1D splines as shown in Ref. [5], where N is the number of grid intervals (proportional to the number of model parameters).

2. Width and depth expansion for increased expressive power

The KAN network generated by the *kanpiler* is compact, without redundant edges, which might limit its expressive power and hinder further fine-tuning. To address this problem, we propose the `expand_width` and `expand_depth` methods to expand the network to become wider and deeper, as shown in Fig. 5(c). The expansion methods initially add zero activation functions, which suffer from zero gradients during training. Therefore, the `perturb` method should be used to perturb these zero functions into nonzero values, making them trainable with nonzero gradients.

IV. KANS TO SCIENCE

Today’s black-box deep neural networks are powerful, but interpreting these models remains challenging. Scientists seek not only high-performing models but also the ability to extract meaningful knowledge from the models. In this section, we focus on enhancing the interpretability of KANs’ scientific purposes. We will explore three levels of knowledge extraction from KANs, from the most basic to the most complex: important features (Sec. IV A), modular structures (Sec. IV B), and symbolic formulas (Sec. IV C).

A. Identifying important features from KANs

Identifying important variables is crucial for many tasks. Given a regression model f where $y \approx f(x_1, x_2, \dots, x_n)$, we aim to assign scores to the input variables to gauge their importance. Liu *et al.* [5] used the function L1 norm, i.e., the integral of the absolute value, which is a commonly used statistical regularization to encourage sparsity, to indicate the importance of edges, but this metric could be problematic as it considers only local information. To address this, we introduce a more effective attribution score which better reflects the importance of variables than the L1 norm. For simplicity, let us assume there are multiplication nodes, so we do not need to differentiate between nodes and subnodes [10]. Suppose we have an L -layer KAN with width $[n_0, n_1, \dots, n_L]$. We define $E_{l,i,j}$ as the standard deviation of the activations on the (l, i, j) edge, and $N_{l,i}$ as the standard deviation of the activations on the (l, i) node, taken across the training dataset. We define $E_{l,i,j}$ as the standard deviation of the activations on the (l, i, j) edge, and $N_{l,i}$ as the standard deviation of the activations on the (l, i) node. We then define the node (attribution) score $A_{l,i}$ and the edge (attribution) score $B_{l,i,j}$. In Ref. [5], we simply defined $B_{l,i,j} = E_{l,i,j}$ and $A_{l,i} = N_{l,i}$. However, this definition fails to account for the later parts of the network; even if a node or an edge has a large norm itself, it may not contribute to the output if the rest of the network is effectively a zero function. Therefore, we now compute the node and edge scores iteratively from the output layer to the input layer. We set all output dimensions to have unit scores, i.e., $A_{L,i} = 1$, $i = 0, 1, \dots, n_L - 1$ [11], and compute scores as follows: For edge $(\ell - 1, i, j)$ (from node i in layer $\ell - 1$ to node j in layer ℓ), we define $(l = L, L - 1, \dots, 1)$

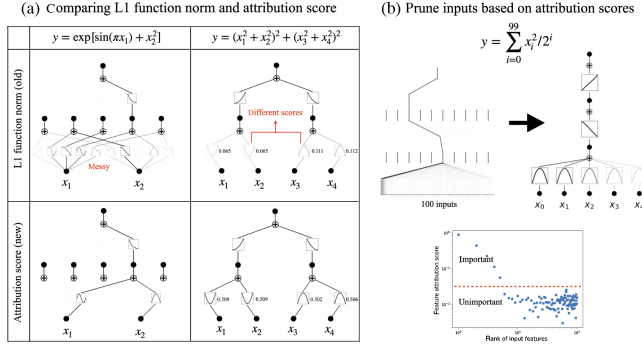


FIG. 6. Identifying important features in KANs. (a) Comparing the attribution score to the L1 norm used in Liu *et al.* [5]. On two synthetic tasks, the attribution score brings more insights than the L1 norm. (b) Attribution scores can be computed for inputs and used for input pruning.

$$B_{l-1,i,j} = A_{l,j} \frac{E_{l,j}}{N_{l+1,j}}, \quad A_{l-1,i} = \sum_{j=0}^{n_l} B_{l-1,i,j}. \quad (11)$$

1. Comparing $E_{l,i,j}$ and $B_{l,i,j}$

We find that $B_{l,i,j}$ provides a more accurate reflection of edge importance. In Fig. 6, we compare KANs trained on two equations $y = \exp[\sin(\pi x_1) + x_2^2]$ and $y = (x_1^2 + x_2^2)^2 + (x_3^2 + x_4^2)^2$ and visualize KANs with importance scores being E (L1 norm) or B (attribution score). For the first equation, attributions scores reveal a cleaner graph than L1 norms, as many active edges in the first layer do not contribute to the final output due to inactive subsequent edges. The attribution score accounts for this problem, resulting in a more meaningful graph. For the second equation $y = (x_1^2 + x_2^2)^2 + (x_3^2 + x_4^2)^2$, we can tell from the symbolic equation that all four variables are equally important. The attribution scores correctly reflect the equal importance of all four variables, whereas the L1 norm incorrectly suggests that x_3 and x_4 are more important than x_1 and x_2 .

2. Pruning inputs based on attribution scores

In real datasets, input dimensionality can be large, but only a few variables may be relevant. To address this, we propose pruning away irrelevant features based on attribution scores so that we can focus on the most relevant ones. Users can apply the `prune_input` to retain only the most relevant variables. For instance, if there are 100 input features ordered by decreasing relevance in the function $y = \sum_{i=0}^{99} x_i^2 / 2^i$, $x_i \in [-1, 1]$, and after training, only the first five features show significantly higher attribution scores, the `prune_input` method will retain only these five features. The pruned network becomes compact and interpretable, whereas the original KAN with 100 inputs is too dense for straightforward interpretation. We remark that

the last two layers are effectively constants and can be pruned.

B. Identifying modular structures from KANs

Although the attribution score provides valuable insights into which edges or nodes are important, it does not reveal modular structures, i.e., how the important edges and nodes are connected. In this section, we aim to uncover modular structures from trained KANs and MLPs by examining two types of modularity: anatomical modularity and functional modularity.

1. Anatomical modularity

Anatomical modularity refers to the tendency for neurons placed close to each other spatially to have stronger connections than those farther apart. Although artificial neural networks lack physical spatial coordinates, introducing the concept of physical space has been shown to enhance interpretability [12,13]. We adopt the neuron swapping method from Refs. [12,13], which shortens connections while preserving the network's functionality. We call the method `auto_swap`. The anatomical modular structure revealed through neuron swapping facilitates easy identification of modules, even visually, for two tasks shown in Fig. 7: (1) multitask sparse parity and (2) hierarchical majority voting. For multitask sparse parity, we have ten input bits $x_i \in \{0, 1\}$, $i = 1, 2, \dots, 10$ and output $y_j = x_{2j-1} \oplus x_{2j}$, $j = 1, \dots, 5$, where $\oplus$ denotes a modulo-2 addition. The task exhibits modularity because each output depends only on a subset of inputs. `auto_swap` successfully identifies modules for both KANs and MLPs, with the KAN discovering simpler modules. For hierarchical majority voting, with nine input bits $x_i \in \{0, 1\}$, $i = 1, \dots, 9$, and the output $y = \text{maj}[\text{maj}(x_1, x_2, x_3), \text{maj}(x_4, x_5, x_6), \text{maj}(x_7, x_8, x_9)]$, where `maj` stands for

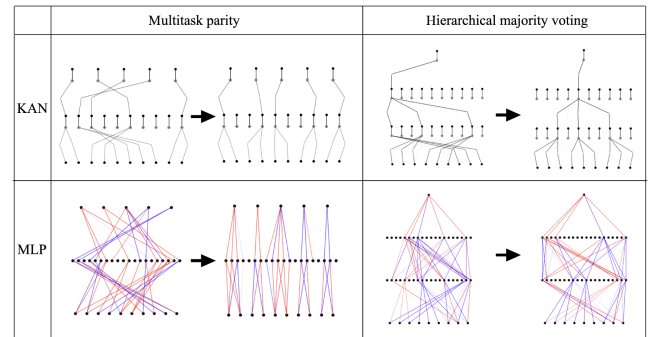


FIG. 7. Inducing anatomical modularity in neural networks through neuron swapping. The approach involves assigning spatial coordinates to neurons and permuting them to minimize the overall connection cost. For two tasks (left, multitask parity; right, hierarchical majority voting), neuron swapping works for KANs (top) in both cases and works for MLPs (bottom) for multitask parity.

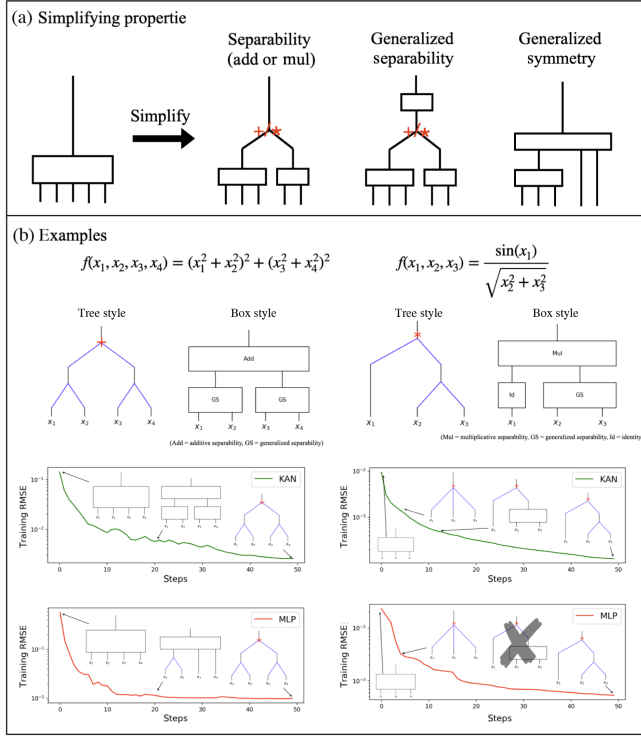


FIG. 8. Detecting functional modularity in KANs. (a) We study three types of functional modularity: separability (additive or multiplicative), general separability, and symmetry. (b) Applying these tests recursively converts a function into a tree. Here the function can be symbolic functions (top), KANs (middle), or MLPs (bottom). Both KANs and MLPs produce correct tree graphs at the end of training but show different training dynamics.

majority voting (output 1 if two or three inputs are 1, otherwise 0). The KAN reveals the modular structure even before `auto_swap`, and the diagram becomes more organized after `auto_swap`. The MLP shows some modular structure from the pattern of the first layer weights, indicating interactions among variables, but the global modular structure remains unclear regardless of `auto_swap`.

2. Functional modularity

Functional modularity pertains to the overall function represented by the neural network. Given an Oracle network where internal details such as weights and hidden layer activations are inaccessible (too complicated to analyze), we can still gather information about functional modularity through forward and backward passes at the inputs and outputs. We define three types of functional modularity [see Fig. 8(a)] based largely on Ref. [14].

Separability. A function f is additively separable if

$$f(x_1, x_2, \dots, x_n) = g(x_1, \dots, x_k) + h(x_{k+1}, \dots, x_n). \quad (12)$$

Note that $[(\partial^2 f)/(\partial x_i \partial x_j)] = 0$ when $1 \leq i \leq k, k+1 \leq j \leq n$. To detect the separability, we can compute

the Hessian matrix $\mathbf{H} \equiv \nabla^T \nabla f$ $\{\mathbf{H}_{ij} = [(\partial^2 f)/(\partial x_i \partial x_j)]\}$ and check for block structure. In practice for the problems of interest to KANs, the input dimension will not be formidable and a greedy search is sufficient. For higher-dimensional problems, we can also use some combinatorial search methods rooted in the graph theory of connected graphs. If $\mathbf{H}_{ij} = 0$ for all $1 \leq i \leq k$ and $k+1 \leq j \leq n$, then we know f is additively separable. For multiplicative separability, we can convert it to additive separability by taking the logarithm

$$\begin{aligned} f(x_1, x_2, \dots, x_n) &= g(x_1, \dots, x_k) \times h(x_{k+1}, \dots, x_n), \\ \log |f(x_1, x_2, \dots, x_n)| &= \log |g(x_1, \dots, x_k)| \\ &\quad + \log |h(x_{k+1}, \dots, x_n)|. \end{aligned} \quad (13)$$

To detect multiplicative separability, we define $\mathbf{H}_{ij} \equiv [(\partial^2 \log |f|)/(\partial x_i \partial x_j)]$ and check for block structure. Users can call `test_separability` to test general separability.

Generalized separability. A function f has generalized separability if

$$f(x_1, x_2, \dots, x_n) = F(g(x_1, \dots, x_k) + h(x_{k+1}, \dots, x_n)). \quad (14)$$

To detect generalized separability, we compute

$$\begin{aligned} \frac{\partial f}{\partial x_i} &= \frac{\partial F}{\partial g} \frac{\partial g}{\partial x_i} \quad (1 \leq i \leq k), \\ \frac{\partial f}{\partial x_j} &= \frac{\partial F}{\partial h} \frac{\partial h}{\partial x_j} \quad (k+1 \leq j \leq n), \\ \frac{\partial f / \partial x_i}{\partial f / \partial x_j} &= \frac{\partial F / \partial g \partial g / \partial x_i}{\partial F / \partial h \partial h / \partial x_j} = \frac{\partial g / \partial x_i}{\partial h / \partial x_j} \\ &= g_{x_i}(x_1, x_2, \dots, x_k) \times \frac{1}{h_{x_j}(x_{k+1}, \dots, x_n)}, \end{aligned} \quad (15)$$

where we have used $[(\partial F)/(\partial g)] = [(\partial F)/(\partial h)]$. Note that $[(\partial f / \partial x_i)/(\partial f / \partial x_j)]$ is multiplicatively separable; it can be detected by the separability test proposed above. Users can call `test_general_separability` to check for additive or multiplicative separability.

Generalized symmetry. A function has generalized symmetry (in the first k variables) if

$$f(x_1, x_2, \dots, x_n) = g(h(x_1, \dots, x_k), x_{k+1}, \dots, x_n). \quad (16)$$

We denote $\mathbf{y} = (x_1, \dots, x_k)$ and $\mathbf{z} = (x_{k+1}, \dots, x_n)$. This property is called generalized symmetry because f retains the same value as long as h is held constant, regardless of individual values of $x_1, \dots, x_k$. We compute the gradient of f with respect to $\mathbf{y}$: $\nabla_{\mathbf{y}} f = [(\partial g)/(\partial h)] \nabla_{\mathbf{y}} h$. Since $(\partial g)/(\partial h)$ is a scalar function, it does not change the direction of $\nabla_{\mathbf{y}} h$.

Thus, the direction of $\widehat{\nabla_y f} \equiv [(\nabla_y f)/(|\nabla_y f|)]$ is independent of $\mathbf{z}$, i.e.,

$$\nabla_{\mathbf{z}}(\widehat{\nabla_y f}) = 0, \quad (17)$$

which is the condition for symmetry. Users can call the `test_symmetry` method to check for symmetries.

Tree converter. The three types of functional modularity form a hierarchy: Symmetry is the most general, general separability is intermediate, and separability is the most specific. Mathematically,

$$\begin{aligned} \text{Separability} &\subset \text{Generalized Separability} \\ &\subset \text{Generalized Symmetry}. \end{aligned} \quad (18)$$

To obtain the maximal hierarchy of modular structures, we apply generalized symmetry detection recursively, forming groups as small as $k=2$ variables and extending to all $k=n$ variables. For example, let us consider an eight-variable function

$$f(x_1, \dots, x_8) = [(x_1^2 + x_2^2)^2 + (x_3^2 + x_4^2)^2]^2 + [(x_5^2 + x_6^2)^2 + (x_7^2 + x_8^2)^2]^2, \quad (19)$$

which has four $k=2$ generalized symmetries involving groups (x_1, x_2) , (x_3, x_4) , (x_5, x_6) , (x_7, x_8) and two $k=2$ generalized symmetries involving groups (x_1, x_2, x_3, x_4) and (x_5, x_6, x_7, x_8) . As such, each $k=4$ group contains two $k=2$ groups demonstrating a hierarchy. For each generalized symmetry, we can also test if the generalized symmetry is further generalized separable or separable. Users can use the method `plot_tree` to obtain the tree graph for a function (the function could be any PYTHON expression, neural networks, etc.). For a neural-network model, users can simply call `model.tree()`. The tree plot can have the style “tree” (by default) or “box.”

Examples. Figure 8(b) provides two examples. When the exact symbolic functions are input to `plot_tree`, the ground truth tree graphs are obtained. We are particularly interested in whether the tree converter works for neural networks. For these simple cases, both KANs and MLPs can find the correct graph if sufficiently trained. Figure 8(b) (bottom) shows the evolution of the tree graphs during KAN and MLP training. It is particularly interesting to see how neural networks gradually learn the correct modular structure. In the first case $f(x_1, x_2, x_3, x_4) = (x_1^2 + x_2^2)^2 + (x_3^2 + x_4^2)^2$, both the KAN and MLP gradually pick up more inductive biases (their intermediate states are different) until they reach the correct structure. In the second case $f(x_1, x_2, x_3) = \sin(x_1)/\sqrt{x_2^2 + x_3^2}$, both models initially detect multiplicative separability for all three variables, showing even higher symmetry than the correct structure. After training progresses, both models “realize” that in order to better fit the data (loss becomes lower), such a

high-symmetry structure can no longer be met and should be relaxed to a less stringent structure. An additional observation is that the KAN has an intermediate structure not found in the MLP. There are two caveats we would like to mention: (1) Results can be seed and/or threshold dependent. (2) All tests rely on second-order derivatives, which may not be robust due to the model being trained only on zero-order information. Adversarial constructions such as $f_\epsilon(x) = f(x) + \epsilon \sin(x/\epsilon)$ could lead to issues, because although $|f_\epsilon(x) - f(x)| \rightarrow 0$ as $\epsilon \rightarrow 0$, $|f'_\epsilon(x) - f'(x)| \rightarrow \infty$ as $\epsilon \rightarrow 0$. Although such extreme cases are unlikely in practice, smoothness is necessary to ensure the success of our methods. We remark that the method is not very sensitive to the threshold, namely, the small constant we set for the zero conditions.

C. Identifying symbolic formulas from KANs

Symbolic formulas are the most informative, as they clearly reveal both important features and modular structures once they are known. In Liu *et al.* [5], the authors showed many examples from which they could extract symbolic formulas with some prior knowledge when needed. With the new tools proposed above (feature importance, modular structures, and symbolic formulas), users can leverage these new tools to easily interact and collaborate with KANs, making symbolic regression easier. We present three tricks below, illustrated in Fig. 9.

1. Trick A: Discover and leverage modular structures

We can first train a general network and probe its modularity. Once the modular structure is identified, we

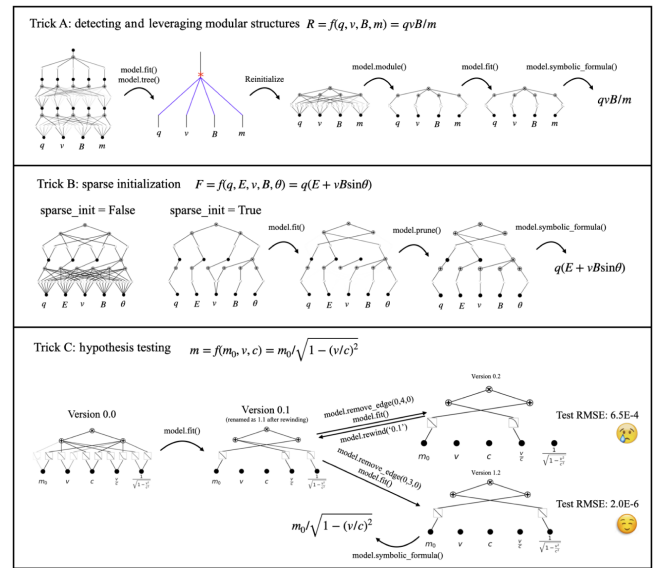


FIG. 9. Three tricks to facilitate symbolic regression. Trick A (top row): detecting and leveraging modular structures. Trick B (middle row): sparse connection initialization. Trick C (bottom row): hypothesis testing.

initialize a new model with this modular structure as inductive biases. For instance, consider the function $f(q, v, B, m) = qvB/m$. We first initialize a large KAN (presumably expressive enough) to fit the dataset to a reasonable accuracy. After training, the tree graph is extracted (see Sec. IV B) from the trained KAN, which shows multiplicative separability. Then we can build the modular structure into a second KAN (see Sec. III B), train it, and then symbolify all 1D functions to derive the formula.

2. Trick B: Sparse initialization

Symbolic formulas typically correspond to KANs with sparse connections [see Fig. 5(b)], so initializing KANs sparsely aligns them better with the inductive biases of symbolic formulas. Otherwise, densely initialized KANs require careful regularization to promote sparsity. Sparse initialization can be achieved by passing the argument “sparse_init=True” to the KAN initializer. For example, for the function $f(q, E, v, B, \theta) = q(E + vB \sin \theta)$, a sparsely initialized KAN closely resembles the final trained KAN, requiring only minor adjustments in training. In contrast, a dense initialization would involve extensive training to remove unnecessary edges. We remark that this initializes only a subset of edges to small random values, with others set to zero. Importantly, these zero-initialized edges are still trainable; they are not frozen and can be activated during training if they prove useful. This ensures that sparse initialization does not rigidly constrain the final structure but instead encourages the network to grow only the necessary complexity. Combining this trick with moderate regularization or hypothesis testing below can help ensure alignment with ground truth structure.

3. Trick C: Hypothesis testing

When faced with multiple reasonable hypotheses, we can try all of them (branching into “parallel universes”) to test which hypothesis is the most accurate and/or simplest. To facilitate hypothesis testing, we build a checkpoint system that automatically saves model versions whenever changes (e.g., training, pruning) are made. For example, consider the function $f(m_0, v, c) = m_0/\sqrt{1 - (v/c)^2}$. We start from a randomly initialized KAN, which has version 0.0. After training, it evolves to version 0.1, where it activates on both $\beta = v/c$ and $\gamma = 1/\sqrt{1 - (v/c)^2}$. Hypothesize that only β or γ might be needed. We first set the edge on γ to zero, and train the model, obtaining a 6.5×10^{-4} test squared root of mean squared error (version 0.2). To test the alternative hypothesis, we want to revert back to the branching point (version 0.1). We call `model.rewind('0.1')`, which rewinds the model back to version 0.1. To indicate that rewind is called, version 0.1 is renamed as version 1.1. Now we set the edge on β to be zero, train the model, obtaining a 2.0×10^{-6} test squared root of mean squared

error (the version becomes 1.2). Comparing versions 0.2 and 1.2 indicates that the second hypothesis is better due to the lower loss given the same complexity (both hypotheses have two nonzero edges).

V. APPLICATIONS

The previous sections primarily focused on regression problems for pedagogical purposes. In this section, we apply KANs to discover physical concepts, such as conserved quantities, Lagrangians, hidden symmetries, and constitutive laws. These examples illustrate how the tools proposed in this paper can be effectively integrated into real-life scientific research to tackle these complex tasks.

A. Discovering conserved quantities

Conserved quantities are physical quantities that remain constant over time. For example, a free-falling ball converts its gravitational potential energy into kinetic energy, while the total energy (the sum of both forms of energy) remains constant (assuming negligible air resistance). Conserved quantities are crucial because they often correspond to symmetries in physical systems and can simplify calculations by reducing the dimensionality of the system. Traditionally, deriving conserved quantities with paper and pencil can be time consuming and demands extensive domain knowledge. Recently, machine learning techniques have been explored to discover conserved quantities [15–20].

We follow the approach Liu *et al.* [16] who derived a differential equation that conserved quantities must satisfy, thus transforming the problem of finding conserved quantities into differential equation solving. They used MLPs to parametrize conserved quantities. We basically follow their procedure but replace MLPs with KANs. To be specific, they consider a dynamical system with the state variable $\mathbf{z} \in \mathbb{R}^d$ governed by the equation $[(d\mathbf{z})/(dt)] = \mathbf{f}(\mathbf{z})$. The necessary and sufficient condition for a function $H(\mathbf{z})$ to be a conserved quantity is that $\mathbf{f}(\mathbf{z}) \cdot \nabla H(\mathbf{z}) = 0$ for all $\mathbf{z}$. For example, in a 1D harmonic oscillator, the phase space is characterized by position and momentum $\mathbf{z} = (x, p)$, and the evolution equation is $d(x, p)/dt = (p, -x)$. The energy $H = \frac{1}{2}(x^2 + p^2)$ is a conserved quantity because $\mathbf{f}(\mathbf{z}) \cdot \nabla H(\mathbf{z}) = (p, -x)(x, p) = 0$. We parametrize H using a KAN, and train it with the loss function $\ell = \sum_{i=1}^N |\mathbf{f}(\mathbf{z}^{(i)}) \cdot \widehat{\nabla} H(\mathbf{z}^{(i)})|^2$ where $\widehat{\nabla}$ is the normalized gradient, and $\mathbf{z}^{(i)}$ is the i th data point uniformly drawn from the hypercube $[-1, 1]^d$.

We choose the 2D harmonic oscillator to test KANs characterized by (x, y, p_x, p_y) . It has three conserved quantities: (1) energy along the x direction $H_1 = \frac{1}{2}(x^2 + p_x^2)$; (2) energy along the y direction $H_2 = \frac{1}{2}(y^2 + p_y^2)$; (3) angular momentum $H_3 = xp_y - yp_x$. We train [4, [0, 2], 1] KANs with three different random seeds, as shown in Fig. 10, which correspond to $H_1, H_2,$

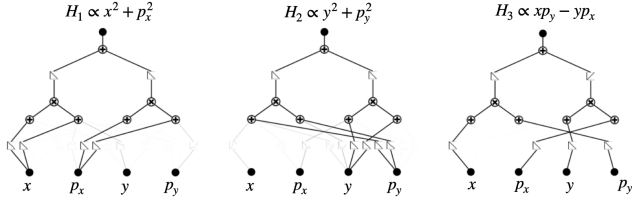


FIG. 10. Using KANs to discover conserved quantities for the 2D harmonic oscillator.

and H_3 , respectively. We remark that we train multiple KANs with different random seeds to discover several conserved quantities. To detect and remove duplicates, we compare ∇H_i across sampled points using cosine similarity. If gradients of two conserved quantities are consistently aligned, we consider them equivalent and prune redundancies.

B. Discovering Lagrangians

In physics, Lagrangian mechanics is a formulation of classical mechanics based on the principle of stationary action. It describes a mechanical system using phase space and a smooth function $\mathcal{L}$ known as the Lagrangian. For many systems, $\mathcal{L} = T - V$, where T and V represent the kinetic and potential energy of the system, respectively. The phase space is typically described by $(\mathbf{q}, \dot{\mathbf{q}})$, where $\mathbf{q}$ and $\dot{\mathbf{q}}$ denote coordinates and velocities, respectively. The equation of motion can be derived from the Lagrangian via the Euler-Lagrange equation $[d/(dt)][(\partial\mathcal{L})/(\partial\dot{\mathbf{q}})] = [(\partial\mathcal{L})/(\partial\mathbf{q})]$, or equivalently,

$$\ddot{\mathbf{q}} = (\nabla_{\dot{\mathbf{q}}} \nabla_{\dot{\mathbf{q}}}^T \mathcal{L})^{-1} [\nabla_{\mathbf{q}} \mathcal{L} - (\nabla_{\mathbf{q}} \nabla_{\dot{\mathbf{q}}}^T \mathcal{L})]. \quad (20)$$

Given the fundamental role of the Lagrangian, an interesting question is whether we can infer the Lagrangian from the data. Following Ref. [21], we train a Lagrangian neural network to predict $\ddot{\mathbf{q}}$ from $(\mathbf{q}, \dot{\mathbf{q}})$. An Lagrangian neural network (LNN) uses an MLP to parametrize $\mathcal{L}(\mathbf{q}, \dot{\mathbf{q}})$ and computes Eq. (20) to predict instant accelerations $\ddot{\mathbf{q}}$. However, LNNs face two main challenges: (1) The training of LNNs can be unstable due to the second-order derivatives and matrix inversion in Eq. (20). (2) LNNs lack interpretability because MLPs themselves are not easily interpretable. We address these issues using KANs.

To tackle the first challenge, we note that the matrix inversion of the Hessian $(\nabla_{\dot{\mathbf{q}}} \nabla_{\dot{\mathbf{q}}}^T \mathcal{L})^{-1}$ becomes problematic when the Hessian has eigenvalues close to zero. To mitigate this, we initialize $(\nabla_{\dot{\mathbf{q}}} \nabla_{\dot{\mathbf{q}}}^T \mathcal{L})$ as a positive definite matrix (or a positive number in 1D). Since $(\nabla_{\dot{\mathbf{q}}} \nabla_{\dot{\mathbf{q}}}^T \mathcal{L})$ is the mass m in classical mechanics and kinetic energy is usually $T = \frac{1}{2} m \dot{\mathbf{q}}^2$, encoding this prior knowledge into KANs is more straightforward than encoding into MLPs (using the kanpiler introduced in Sec. III C). The kanpiler can convert the symbolic formula T into a KAN (as shown in Fig. 11).

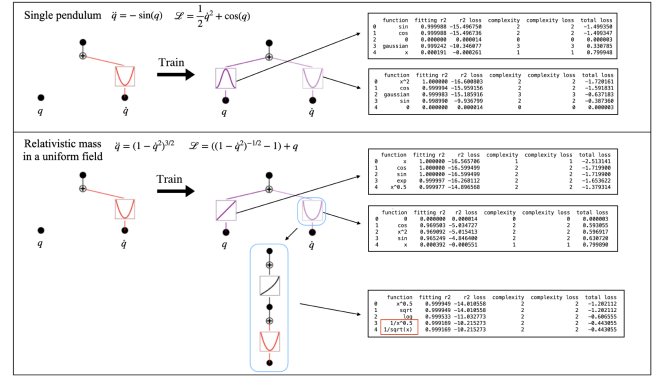


FIG. 11. Use KANs to learn Lagrangians for the single pendulum (top) and a relativistic mass in a uniform field (bottom).

We use this converted KAN for initialization and continue training, resulting in much greater stability compared to random initialization. After training, symbolic regression can be applied to each edge to extract out symbolic formulas, addressing the second challenge.

We show two 1D examples in Fig. 11, a single pendulum and a relativistic mass in a uniform field. The compiled KANs are displayed on the left, with edges on $\dot{q}$ displaying quadratic functions and edges on q as zero functions.

1. Single pendulum

The $\dot{q}$ part remains a quadratic function $T(\dot{q}) = \frac{1}{2} \dot{q}^2$, while the q part learns to be a cosine function, as $V(q) = 1 - \cos(q)$. In Fig. 11 (top), the results from `suggest_symbolic` display the top five functions that best match the splines, considering both fitness and simplicity. As expected, the cosine and the quadratic function appear at the top of the lists.

2. Relativistic mass in a uniform field

After training, the kinetic energy part deviates from $T = \frac{1}{2} \dot{q}^2$ because, for a relativistic particle, $T_r = (1 - \dot{q}^2)^{-1/2} - 1$. In Fig. 11 (bottom), symbolic regression successfully finds $V(q) = q$, but fails to identify T_r due to its compositional nature, as our symbolic regression searches only for simple functions. By assuming the first function composition is quadratic, we create another $[1, 1, 1]$ KAN to fit T_r , and set the first function to be the quadratic function using `fix_symbolic` and train only the second learnable function. After training, we see that the ground truth $x^{-1/2}$ appears among the top five candidates. However, $x^{1/2}$ fits the spline slightly better, as indicated by a higher R -squared value. This suggests that symbolic regression is sensitive to noise (due to imperfect learning) and prior knowledge is crucial for correct judgment. For instance, knowing that kinetic energy should diverge as the velocity approaches the speed of light helps confirm $x^{-1/2}$ as the correct term, since $x^{1/2}$ does not exhibit the expected divergence.

C. Discovering hidden symmetry

Philip Anderson famously argued that “it is only slightly overstating that case to say that physics is the study of symmetry,” emphasizing how the discovery of symmetries has been invaluable for both deepening our understanding and solving problems more efficiently. However, symmetries are sometimes not manifest but hidden, revealed only by applying some coordinate transformation. For example, after Schwarzschild discovered his eponymous black-hole metric, it took 17 years for Painlevé, Gullstrand, and Lemaître to uncover its hidden translational symmetry. They demonstrated that the spatial sections could be made translationally invariant with a clever coordinate transformation, thereby deepening our understanding of black holes [22]. Liu and Tegmark [23] showed that the Gullstrand-Painlevé transformation can be discovered by training an MLP in minutes. However, they did not get extremely high precision (i.e., machine precision) for the solution. We attempt to revisit this problem using KANs.

Suppose there is a Schwarzschild black hole in spacetime (t, x, y, z) with mass $2M = 1$ centered at $x = y = z = 0$ with a radius $r_s = 2M = 1$. The Schwarzschild metric describes how space and time distorts around it:

$$g_{\mu\nu} = \begin{pmatrix} 1 - \frac{2M}{r} & 0 & 0 & 0 \\ 0 & -1 - \frac{2Mx^2}{(r-2M)r^2} & -\frac{2Mxy}{(r-2M)r^2} & -\frac{2Mxz}{(r-2M)r^2} \\ 0 & -\frac{2Mxy}{(r-2M)r^2} & -1 - \frac{2My^2}{(r-2M)r^2} & -\frac{2Myz}{(r-2M)r^2} \\ 0 & -\frac{2Mxz}{(r-2M)r^2} & -\frac{2Myz}{(r-2M)r^2} & -1 - \frac{2Mz^2}{(r-2M)r^2} \end{pmatrix}. \quad (21)$$

Applying the Gullstrand-Painlevé transformation $t' = t + 2M(2u + \ln[(u-1)/(u+1)])$, $u \equiv \sqrt{r/(2M)}$, $x' = x$, $y' = y$, $z' = z$, the metric in the new coordinates becomes

$$g'_{\mu\nu} = \begin{pmatrix} 1 - \frac{2M}{r} & -\sqrt{\frac{2M}{r}} \frac{x}{r} & -\sqrt{\frac{2M}{r}} \frac{y}{r} & -\sqrt{\frac{2M}{r}} \frac{z}{r} \\ -\sqrt{\frac{2M}{r}} \frac{x}{r} & -1 & 0 & 0 \\ -\sqrt{\frac{2M}{r}} \frac{y}{r} & 0 & -1 & 0 \\ -\sqrt{\frac{2M}{r}} \frac{z}{r} & 0 & 0 & -1 \end{pmatrix}, \quad (22)$$

which exhibits translation invariance in the spatial section (the lower right 3×3 block is the Euclidean metric). Liu and Tegmark [23] used an MLP to learn the mapping from (t, x, y, z) to (t', x', y', z') . Defining the Jacobian matrix $\mathbf{J} \equiv \{[\partial(t', x', y', z')]/[\partial(t, x, y, z)]\}$, $\mathbf{g}$ is transformed into $\mathbf{g}' = \mathbf{J}^{-T} \mathbf{g} \mathbf{J}^{-1}$. We take the bottom right 3×3 block of $\mathbf{g}'$ and take its difference to the Euclidean metric to obtain the MSE loss. The loss is minimized by doing gradient descents on the MLP. To make things simple, they assume knowing

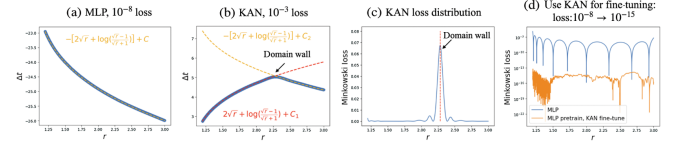


FIG. 12. Rediscovering the hidden symmetry of the Schwarzschild black hole with MLPs and KANs. (a) $\Delta t(r)$ learned by the MLP is a globally smooth solution. (b) $\Delta t(r)$ learned by the KAN is a domain-wall solution. (c) The KAN shows a loss spike at the domain wall. (d) A KAN can be used to fine-tune the MLP solution close to machine precision.

$x' = x, y' = y, z' = z$, and use only an MLP (one input and one output) to predict the temporal difference $\Delta t(r) = t' - t = 2M(2u + \ln[(u-1)/(u+1)])$, $u \equiv \sqrt{r/(2M)}$ from the radius r .

1. MLP and KAN find different solutions

We trained both an MLP and a KAN to minimize this loss function, with results shown in Fig. 12. Since the task has one input dimension and one output dimension, the KAN effectively reduces to a spline. We originally expected KANs to outperform MLPs, because splines are known to be superior in low-dimensional settings [24]. However, while an MLP can achieve a 10^{-8} loss, the KAN gets stuck at a 10^{-3} loss despite grid refinements. It turned out that the KAN and MLP learned two different solutions: While the MLP found a globally smooth solution [Fig. 12(a)], the KAN learned a domain-wall solution [Fig. 12(b)]. The domain-wall solution has a singular point that separates the whole curve into two segments. The left segment learns $\Delta t(r)$ correctly, while the right segment learns $-\Delta t(r)$, which is also a valid solution but differs from the left segment by a minus sign. There is a loss spike appearing at the singular point [Fig. 12(c)]. One might consider this a feature of KANs because domain-wall solutions are prevalent in nature. However, if one considers this a flaw, KANs can still obtain globally smooth solutions by adding regularizations (to reduce spline oscillations) or experimenting with different random seeds (roughly one out of three random seeds finds a global smooth solution).

2. KANs can achieve extreme precision

Although the MLP finds the globally smooth solution and achieves a 10^{-8} loss, the loss is still far from machine precision. We found that neither longer training nor increasing the MLP’s size significantly reduced the loss. Therefore, we turned to KANs, which, as splines in 1D, can achieve arbitrary accuracy by refining the grid (given infinite data). We first used the MLP as a teacher, generating supervised pairs (x, y) to train the KAN to fit the supervised data. This way, the KAN is initialized to a globally smooth solution. We then iteratively refined the KAN by increasing the number of grid intervals to 1000.

In the end, the fine-tuned KANs achieved a loss of 10^{-15} , close to machine precision [Fig. 12(d)].

D. Learning constitutive laws

A constitutive law defines the behavior and properties of a material by modeling how it responds to external forces or deformations. One of the simplest forms of constitutive law is Hooke's law [25], which relates the strain and stress of elastic materials linearly. Constitutive laws encompass a wide range of materials, including elastic materials [26,27], plastic materials [28], and fluids [29]. Traditionally, these laws were derived from first principles based on theoretical and experimental studies [30–33]. Recent advancements, however, have introduced data-driven approaches that leverage machine learning to discover and refine these laws from dedicated datasets [34–37]. We follow the standard notations and experimental setups in the elasticity part of NCLaw [36] and define the constitutive law as a parametrized function $\mathcal{E}_\theta(\mathbf{F}) \rightarrow \mathbf{P}$, where $\mathbf{F}$ denotes the deformation tensor, $\mathbf{P}$ the first Piola-Kirchhoff stress tensor, and θ the parameters in the constitutive law.

Many isotropic materials have linear constitutive laws when deformation is small:

$$\mathbf{P}_I = \mu(\mathbf{F} + \mathbf{F}^T - 2\mathbf{I}) + \lambda(\text{Tr}(\mathbf{F}) - 3)\mathbf{I}. \quad (23)$$

However, when deformation gets larger, nonlinear effects start to kick in. For example, a Neo-Hookean material has the following constitutive law:

$$\mathbf{P} = \mu(\mathbf{F}\mathbf{F}^T - \mathbf{I}) + \lambda \log[\det(\mathbf{F})]\mathbf{I}, \quad (24)$$

where μ and λ are the so-called Lamé parameters determined by the so-called Young's modulus Y and Poisson ratio ν as $\mu = \{Y/[2(1 + \nu)]\}$, $\lambda = \{Y\nu/[(1 + \nu)(1 - 2\nu)]\}$. For simplicity, we choose $Y = 1$ and $\nu = 0.2$, hence, $\mu = \frac{5}{12} \approx 0.42$ and $\lambda = \frac{5}{18} \approx 0.28$.

Assuming we are working with Neo-Hookean materials, and our goal is to use KANs to predict the $\mathbf{P}$ tensor from the $\mathbf{F}$ tensor. Suppose we do not know they are neo-Hookean materials, but we have the prior knowledge that the linear constitutive law is approximately valid for small deformation. Because of symmetries, it suffices to demonstrate that we can accurately predict P_{11} and P_{12} from the nine matrix elements of $\mathbf{F}$. We want to compile linear constitutive laws into KANs, which are $P_{11} = 2\mu(F_{11} - 1) + \lambda(F_{11} + F_{22} + F_{33} - 3)$ and $P_{12} = \mu(F_{12} + F_{21})$. We want to extract Neo-Hookean laws from trained KANs, which are $P_{11} = \mu(F_{11}^2 + F_{12}^2 + F_{13}^2 - 1) + \lambda \log[\det(\mathbf{F})]$ and $P_{12} = \mu(F_{11}F_{21} + F_{12}F_{22} + F_{13}F_{23})$. We generate a synthetic dataset by sampling F_{ij} independently from $U[\delta_{ij} - w, \delta_{ij} + w]$ ($w = 0.2$) and using the Neo-Hookean constitutive law to compute $\mathbf{P}$. Our interaction with KANs is illustrated in Fig. 13. In both cases, we successfully figured

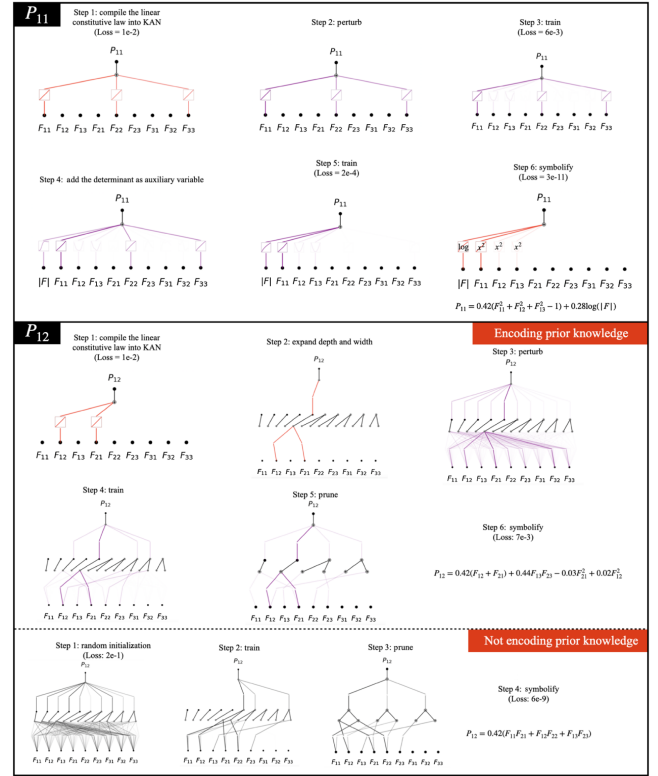


FIG. 13. Discovering constitutive laws (relations between the pressure tensor $\mathbf{P}$ and the strain tensor $\mathbf{F}$) with KANs by interacting with them. Top: predicting the diagonal element P_{11} . Bottom: predicting the off-diagonal element P_{12} .

out the true symbolic formulas in the end, with the aid of some inductive biases. However, the key takeaway is not that we can rediscover the exact symbolic formulas—given that prior knowledge skews the process—but rather in real-world scenarios, where the answers are unknown and users can make guesses based on prior knowledge, the PYKAN package makes it easy to test or incorporate prior knowledge.

1. Predicting P_{11}

In step 1, we compile the linear constitutive law $P_{11} = 2\mu(F_{11} - 1) + \lambda(F_{11} + F_{22} + F_{33} - 3)$ to a KAN using kanpiler, resulting in a 10^{-2} loss. In step 2, we perturb the KAN so that it becomes trainable (indicated by the color change from red to purple; red denotes a purely symbolic part, while purple indicates that both symbolic and spline parts are active). In step 3, we train the perturbed model until convergence, giving a 6×10^{-3} loss. In step 4, assuming that the determinant is a key auxiliary variable, we use `expand_width` (for the KAN) and `augment_input` (for the dataset) to include the determinant $|\mathbf{F}|$. In step 5, we train the KAN until convergence, giving a 2×10^{-4} loss. In step 6, we symbolify the KAN to obtain a symbolic formula $P_{11} = 0.42(F_{11}^2 + F_{12}^2 + F_{13}^2 - 1) + 0.28 \log(|\mathbf{F}|)$, which achieves a 3×10^{-11} loss.

2. Predicting P_{12}

We experimented with and without encoding the linear constitutive law as prior knowledge. With prior knowledge: In step 1, we compile the linear constitutive law to a KAN, resulting in a loss of 10^{-2} . We then perform a series of operations, including expand (step 2), perturb (step 3), train (step 4), prune (step 5), and finally symbolic (step 6). The influence of prior knowledge is evident, as the final KAN identifies only minor correction terms to the linear constitutive law. The final KAN is symbolified as $P_{12} = 0.42(F_{12} + F_{21}) + 0.44F_{13}F_{23} - 0.03F_{21}^2 + 0.02F_{12}^2$, which yields a 7×10^{-3} loss, only slightly better than the linear constitutive law. Without prior knowledge: In step 1, we randomly initialize the KAN model. In step 2, we train the KAN with regularization. In step 3, we prune the KAN to be a more compact model. In step 4, we symbolify the KAN, yielding $P_{12} = 0.42(F_{11}F_{21} + F_{12}F_{22} + F_{13}F_{23})$, which closely matches the exact formula, achieving a 6×10^{-9} loss. Comparing the two scenarios—one with and one without prior knowledge—reveals a surprising outcome: In this example, prior knowledge appears harmful, possibly because the linear constitutive law is probably near a (bad) local minimum which is hard for the model to escape. Indeed, the Neo-Hookean laws are nonlinear and very different from the linear ansatz. However, we should probably not randomly extrapolate this conclusion to more complicated tasks and larger networks. For more complicated tasks, finding a local minimum via gradient descent might be challenging enough, making an approximate initial solution desirable. Additionally, larger networks might be sufficiently overparametrized to eliminate bad local minima, ensuring that all local minima are global and interconnected.

VI. RELATED WORKS

KANs, inspired by KART, were recently proposed by Liu *et al.* [5]. Although the connection between KART and networks has long been deemed irrelevant [38], Liu *et al.* generalized the original two-layer network to arbitrary depths and demonstrated their promise for science-oriented tasks given their accuracy and interpretability. Subsequent research has explored the application of KANs across various domains, including graphs [39–42], partial differential equations [43,44] and operator learning [44–47], tabular data [48], time series [49–52], human activity recognition [53,54], neuroscience [55,56], quantum science [57–59], computer vision [60–65], kernel learning [66], nuclear physics [67], electrical engineering [68], biology [69], and high-energy physics [70]. Liu *et al.* used B -splines to parametrize 1D functions, and other research has explored various activation functions, including the wavelet [64,71], radial basis function [72], Fourier series [73], finite basis [74,75], Jacobi basis functions [76], polynomial basis functions [77], and rational functions [78].

Other techniques for KANs have also been proposed including regularization [79], Kansformer (combining transformer and KAN) [80], adaptive grid update [81], initialization schemes [82], federated learning [83], and convolutional KANs [65]. There have been ongoing debates regarding whether KANs really outperform other neural networks (especially MLPs) on various domains [61,63,84–86], which suggests that while KANs show promise for machine learning tasks, further development is needed to surpass state-of-the-art models.

A. Machine learning for physical laws

A major goal for KANs is to aid in the discovery of new physical laws from the data. Previous research has shown that machine learning can be used to learn various types of physical laws, including equations of motion [87–90], conservation laws [15–20], symmetries [23,91,92], phase transitions [93,94], Lagrangians and Hamiltonians [21,95], symbolic regression [96–99], etc. However, making neural networks interpretable often requires domain-specific knowledge, limiting their generality. We hope that KANs will evolve into universal foundation models for physical discoveries.

Mechanistic interpretability seeks to understand how neural networks operate in a fundamental level [12,100–108]. Some research in this area focuses on designing models that are inherently interpretable [106] or proposing training methods that explicitly promote interpretability [12]. KANs fall into this category since the Kolmogorov-Arnold theorem decomposes a high-dimensional function into a collection of 1D functions, which are significantly easier to interpret than high-dimensional functions.

VII. DISCUSSION

KAN interpolates between software 1.0 and 2.0: The key difference between KANs and other neural networks (software 2.0, a term coined by Andrej Karpathy) lies in their greater interpretability, which allows for manipulation by users, similar to traditional software (software 1.0). However, KANs are not entirely traditional software, as they have (1) learnability (good), enabling them to learn new things from data, and (2) reduced interpretability (bad) as they become less interpretable and controllable as the network scales increase. Figure 14(a) visualizes the position of software 1.0, software 2.0, and KANs on the interpretability-learnability plane, illustrating how KANs can balance the trade-offs between these two paradigms. The goal of this paper is to propose various tools that make KANs more like software 1.0, while leveraging the learnability of software 2.0.

Interpretability: Although the learnable univariate functions in KANs are more interpretable than weight matrices in MLPs, scalability remains a challenge. As KAN models scale up, even if all spline functions are interpretable

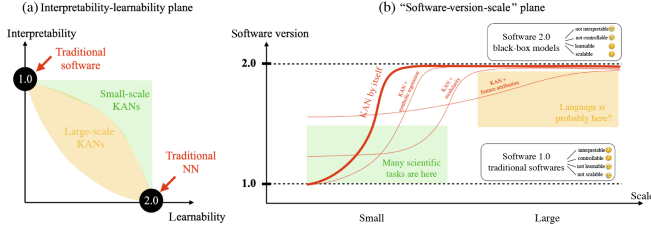


FIG. 14. KAN interpolates between software 1.0 and 2.0. (a) KANs strike a balance between interpretability (software 1.0) and learnability (software 2.0). (b) KANs’ Pareto frontier on the interpretability-scale plane. The amount of interpretation we can get from KANs depends on problem scales and interpretability methods.

individually, it becomes increasingly difficult to manage the combined output of these 1D functions. Consequently, a KAN may remain only interpretable when the network scale is relatively small [Fig. 14(b), thick red line]. It is important to note that interpretability depends on both intrinsic factors (related to the model itself) and extrinsic factors (related to interpretability methods). Advanced interpretability methods should be able to handle interpretability at various levels. For example, by interpreting KANs with symbolic regression, modularity discovery and feature attribution [Fig. 14(b), thin red lines], the Pareto frontier of interpretability versus scale extends beyond what a KAN alone can achieve. A promising direction for future research is to develop more advanced interpretability methods that can further push the current Pareto frontiers.

Extensions and limitations: In Appendix A, we apply KANs to high-dimensional or challenging scientific problems to showcase their ability to solve complex problems. In Appendix B, we showcase some of KANs’ failure modes and ways to avoid them.

Future work: This paper introduces a framework that integrates KANs with scientific knowledge, focusing primarily on small-scale, physics-related examples. Moving forward, two promising directions include applying this framework to larger-scale problems and extending it to other scientific disciplines beyond physics.

ACKNOWLEDGMENTS

We would like to thank Yizhou Liu, Di Luo, Akash Kundu, and many GitHub users for fruitful discussions and constructive suggestions. We extend special thanks to GitHub user Blealtan for making public their work on making KANs efficient. Z. L. and M. T. are supported by IAIFI through NSF Grant No. PHY-2019786.

APPENDIX A: REAL-WORLD EXPERIMENTS

To testify the effectiveness of KANs on real-world scientific problems and high-dimensional problems, we add experiments regarding image fitting, partial differential equation (PDE) solving, and Modified National Institute of

Standards and Technology (MNIST) database classification in the following subsections.

1. Image fitting

a. Fitting images

We task KANs with three images: (1) The cameraman picture is the standard picture for the image fitting task. (2) The turbulence profile is taken from PDEBENCH [109] demonstrating high-frequency and fractal behavior typical in scientific computing. (3) Van Gogh’s *The Starry Night* is quite challenging because it contains fine-grained details as well. In addition to MLPs, We compare KANs with these stronger baselines: (i) MLP with random Fourier features (MLP_RFF). Before feeding input coordinates $\mathbf{x} \equiv (x, y)$ to the MLP, we first augment them into a higher-dimensional feature space $\Phi(\mathbf{x}) = [\mathbf{x}, \Phi_1(\mathbf{x}), \dots, \Phi_{N_f}(\mathbf{x})]$, where $\Phi_i(\mathbf{x}) = [\cos(\mathbf{s}_i \cdot \mathbf{x}), \sin(\mathbf{s}_i \cdot \mathbf{x})]$, $i = 1, \dots, N_f$, and $\mathbf{s}_i \sim \mathcal{N}(0, s^2)$ (s controls the frequency bias). We choose $N_f = 50$ and $s = 3, 30$. (ii) SIREN [110] uses sines as activation functions in MLPs and uses large initialization for the first layer (effectively creating high-frequency features). To compare KANs and baselines as fairly as possible, we try two control strategies (Same shape or same number of parameters) and report both performance [measured by peak signal-to-noise ratio (PSNR)] and efficiency (wall time). All methods are listed below in Table I. For all baseline models, 1 means their width is the same as KAN 1, while 2 means their number of parameters is (approximately) the same as KAN 1 ($\sqrt{G}$ times wider, where $G = 10$ is the grid size used in KAN 1). We also explore KAN 2, which uses a finer grid ($G = 100$ instead of $G = 10$) for the first layer only (inspired by the idea of random Fourier features in the input layer). The whole image is treated as the training set and there is no test set. All models are trained with the ADAM optimizer for 15 000 steps with learning rate decay (5000 steps for learning rate 10^{-3} , 10^{-4} , and 10^{-5}), with batch size 1024, on a V100 GPU.

We list PSNR and training wall time in Table I, and fitted images in Fig. 15. We have a few observations from the results: (1) KANs are comparable to or even outperform baseline methods (including SIREN) in terms of PSNR, however, with more training time. (2) Having random features in the inputs is useful for MLPs, especially high-frequency random features ($s = 30$ outperforms $s = 3$). We may also understand KANs’ superior performance as being good at generating random features in early layers. By changing the grid size in the first layer from $G = 10$ to $G = 100$ (KAN 2), PSNR significantly increases with little additional overhead in training time. We show all three examples in Fig. 15.

2. Partial differential equations

In this section, we test more PDE examples, showing that KANs can achieve reasonable performance for more complicated PDEs. However, we want to note that

TABLE I. Image fitting: comparing various methods.

Method	Width	PSNR $\uparrow$			Training wall time (s) $\downarrow$		
		Cam	Turb	Star	Cam	Turb	Star
KAN 1	[2, 128, 128, 128, 128, 1] $G = [10, 10, 10, 10, 10]$	32.06	62.45	50.55	1800	1721	1715
KAN 2	[2, 128, 128, 128, 128, 1] $G = [100, 10, 10, 10, 10]$	45.76	82.68	71.82	1809	1734	1727
MLP 1	[2, 128, 128, 128, 128, 1]	20.76	27.18	18.28	162	92	91
MLP 2	[2, 404, 404, 404, 404, 1]	22.09	33.14	18.96	182	110	110
SIREN1	[2, 128, 128, 128, 128, 1]	27.34	49.51	29.88	254	226	232
SIREN2	[2, 404, 404, 404, 404, 1]	30.79	51.94	53.05	407	400	404
MLP_RFF 1 ($s = 3$)	[2, 128, 128, 128, 128, 1]	22.17	33.58	19.19	176	96	96
MLP_RFF 2 ($s = 3$)	[2, 404, 404, 404, 404, 1]	24.73	42.57	22.00	192	117	118
MLP_RFF 1 ($s = 30$)	[2, 128, 128, 128, 128, 1]	23.92	43.07	22.68	174	96	97
MLP_RFF 2 ($s = 30$)	[2, 404, 404, 404, 404, 1]	26.26	46.43	28.62	195	117	121

KANs are slightly slower than MLPs to train in terms of wall time, despite their smaller number of parameters. Also, both the LBFGS optimizer and the grid extension technique are required to achieve the theoretical scaling law, but in

practice, people use the ADAM optimizer and do not need a grid extension for MLPs, which we explore below.

In the main text, we showed KANs' superior performance over MLPs for solving a 2D Poisson equation with a

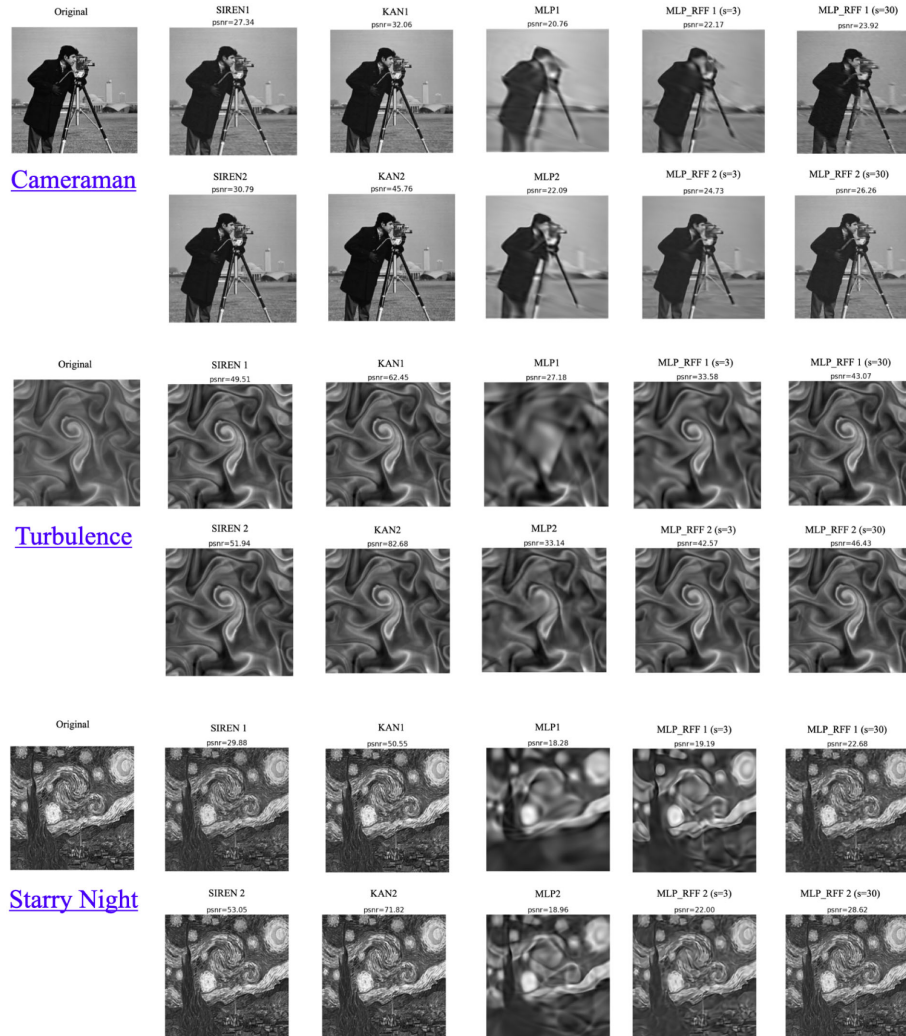


FIG. 15. Comparing various methods for fitting the picture of cameraman (top), turbulent flow (middle, from PDEBENCH [109]), and Van Gogh's *The Starry Night* (bottom).

TABLE II. Comparing various methods for solving 2D Poisson equations. All MLPs (including MLP_RFFs) have shapes [2,128,128,128,1].

Method	$n = 1$		$n = 2$		$n = 4$	
	l_2 error ↓	Time (s) ↓	l_2 error ↓	Time (s) ↓	l_2 error ↓	Time (s) ↓
MLP	0.003	28	0.027	19	0.553	19
MLP (10× training)	0.001	184	0.328	184	0.022	202
MLP_RFF ($s = 3$)	0.001	254	0.001	233	0.084	232
MLP_RFF ($s = 30$)	1.000	250	0.999	242	0.934	249
KAN [2,10,1] $G = 10$	0.006	28	0.135	29	0.729	24
KAN [2,10,1] $G = 20$	0.221	33	0.082	33	0.295	34
KAN [2,100,1] $G = 10$	0.001	51	0.006	52	0.099	51
KAN [2,100,1] $G = 20$	0.326	72	0.135	71	0.090	74
KAN [2,10,10,10,1] $G = 10$	0.012	89	0.117	95	0.576	92
KAN [2,10,10,10,1] $G = 20$	0.995	127	0.993	130	0.982	125

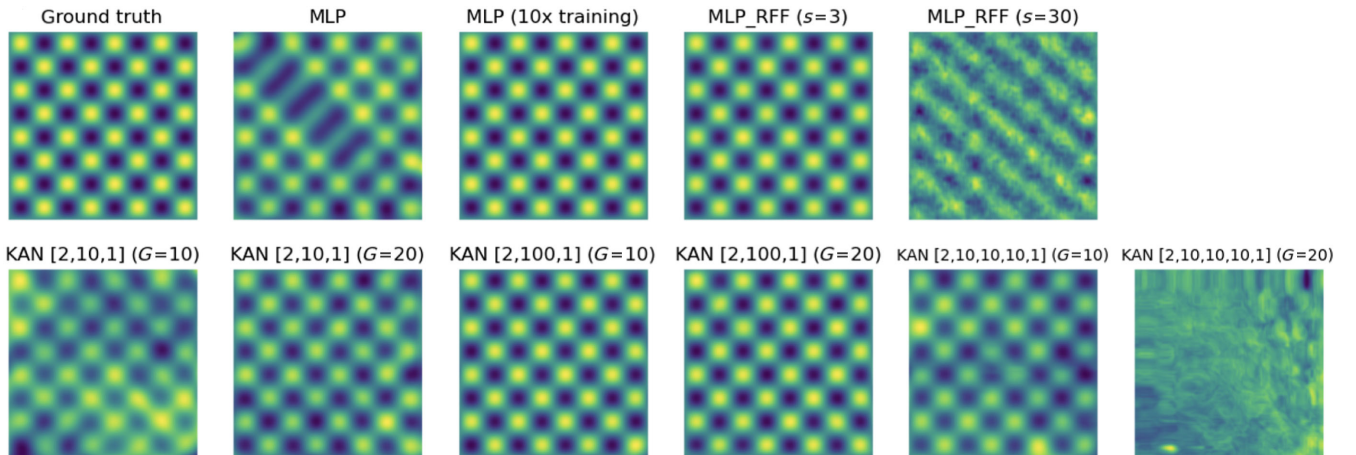
smooth solution. To really understand the capabilities and limitations of KANs, we test KANs by taking three (relatively more) challenging PDEs, as suggested by reviewers: (1) Poisson equations with high-frequency solutions to test KANs' ability to model high-frequency modes, (2) Allen-Cahn equation to test KANs' ability to model temporal phenomenon and capture sharp transitions, and (3) Darcy flow to test KANs' ability to model random structures (e.g., Darcy flow can be used to model porous media). The goal of this section is to show that KANs (as they are) can achieve reasonable performance for these challenging PDEs, rather than attempting to establish state of the art performance. Indeed, PDE modeling with neural networks is a huge field, and many techniques (e.g., adaptive weights [111], causality training [112], the gating mechanism in PIRATENET [113]) have been developed that can be very useful to improve KANs in the future.

A remark on the problem setup: In the main text, we used the LBFGS optimizer and the grid extension technique to

achieve the theoretical scaling law. However, it is more common to use the ADAM optimizer, and the grid extension technique is too specific for KANs. To make fair comparisons in typical user cases, below we use the ADAM optimizer and do not use grid extension (the grid size is fixed once chosen) to avoid the possibility of any of our algorithmic choices favoring KANs. That said, this does not mean one should avoid using these tricks in practice when the goal is to optimize results rather than make fair comparisons. We note that KAN's training wall time is much reduced from the last version because we now disable the symbolic front (which takes up most of the training time but is unnecessary for PDE cases).

a. Poisson equation with high-frequency solutions

To test KANs' ability to approximate high-frequency PDE solutions, we revisit the Poisson equation in the main text but impose high-frequency solutions. To be specific, we consider the Poisson equation

FIG. 16. Comparing various methods on solving the 2D Poisson equation with a high-frequency solution ($n = 4$).

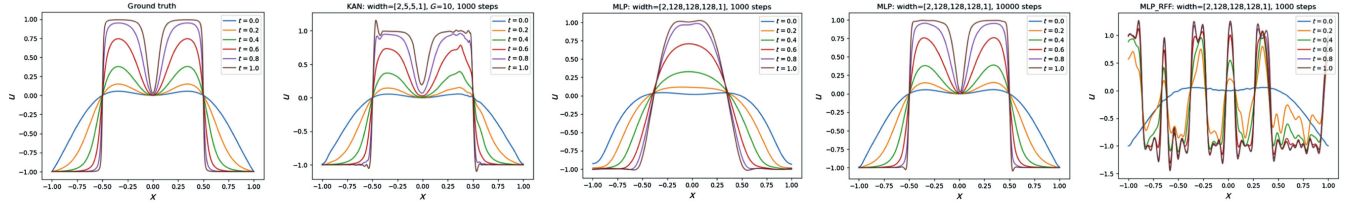


FIG. 17. Solving the 1D Allen-Cahn equation.

$$u_{xx} + u_{yy} = f, \quad \Omega \in [-1, 1]^2 \quad (\text{A1})$$

with zero boundary condition $[u(x, -1) = u(x, 1) = u(-1, y) = u(1, y) = 0]$ and $f = -2n^2\pi^2 \sin(n\pi x) \times \sin(n\pi y)$, which has the solution $u(x, y) = \sin(n\pi x) \times \sin(n\pi y)$. We train our models (listed in Table II) using ADAM optimizers with a learning rate 10^{-3} for 1000 steps except for 10000 steps for MLP (10 $\times$ training). The training loss is the physics-informed neural networks (PINN) loss $\text{loss}_{\text{PDE}} = \alpha \text{loss}_i + \text{loss}_b := \alpha(1/n_i) \sum_{i=1}^{n_i} |u_{xx}(z_i) + u_{yy}(z_i) - f(z_i)|^2 + (1/n_b) \sum_{i=1}^{n_b} u^2$, where we use loss_i to denote the interior loss discretized and evaluated by a uniform sampling of $n_i = 51^2$ points $z_i = (x_i, y_i)$ inside the domain, and similarly we use loss_b to denote the boundary loss discretized and evaluated by a uniform sampling of $n_b = 51$ points on the boundary. $\alpha = 0.01$ is the hyperparameter balancing the effect of the two terms. It is clear that larger n means the solution is more high frequency and hence more challenging. We visualize the predictions by the models in Fig. 16 and list their relative ℓ_2 error and training wall time in Table II. We have a few observations: (1) All the models, i.e., KANs, MLPs, and MLP_RFFs (MLP with random Fourier features) can achieve qualitatively good predictions given proper hyperparameters. (2) High-frequency Fourier features in MLP_RFFs can be harmful to training. Similarly, KANs become unstable with large grid sizes and/or depths. This is in contrast to the results in image fitting in Appendix A. Our explanation is that PINN losses have quite sharp loss landscapes, and adding high-frequency features will only make things worse. (3) KANs have slightly higher training time than MLPs, which is due to the recursive evaluations of splines, and derivatives in the PINN objective makes this problem even more severe. As we will see below, the observations drawn in this example apply to the two equations below as well.

b. Allen-Cahn equation

To test KANs' ability to solve temporal PDEs and model phase transitions, we consider the one-dimensional Allen-Cahn equation with the periodic boundary conditions and the quartic double-well potential energy formulated as

$$u_t - 0.0001u_{xx} + 5(u^3 - u) = 0, \quad x \in [-1, 1], t \in [0, 1],$$

$$u(x, 0) = x^2 \cos(\pi x) \equiv u_0(x),$$

$$u(-1, t) = u(1, t),$$

$$u_x(-1, t) = u_x(1, t). \quad (\text{A2})$$

Since we do not have an exact solution, a reference solution is obtained via the direct Euler forward method (1000 mesh points in space and in time). Although this temporal equation seems innocuous, using standard PINN training (using MLPs) can lead to a problem; the solution would collapse to a zero solution very quickly in time. To solve this issue, Ref. [112] proposes causal training where the temporal domain is divided into several blocks. Each block corresponds to a separate PINN, and these PINNs are trained sequentially in time, where the previous block is used to initialize the next block. Each block covers $\Delta t = 0.1$, so there are ten blocks in total. The training loss is the PINN loss $\text{loss}_{\text{PDE}} = \alpha_i \text{loss}_i + \alpha_b \text{loss}_b + \alpha_t \text{loss}_t := \alpha_i(1/n_i) \sum_{i=1}^{n_i} |u_t - 0.0001u_{xx} + 5(u^3 - u)|^2 + \alpha_b(1/n_b) \sum_{i=1}^{n_b} (u(1, t_i) - u(-1, t_i))^2 + [u_x(1, t_i) - u_x(-1, t_i)]^2 + \alpha_t(1/n_t) \sum_{i=1}^{n_t} (u(x_i, 0) - u_0(x_i))^2$, where we use loss_i to denote the interior loss discretized and evaluated by a uniform sampling of $n_i = 51^2$ points $z_i = (x_i, y_i)$ inside the domain, and similarly, we use loss_b to denote the boundary loss discretized and evaluated by a uniform sampling of $n_b = 51$ points on the boundary. loss_t is used to denote the initial profile discretized and evaluated by a uniform sampling of $n_t = 51$ points on the boundary. We choose $\alpha_i = 1, \alpha_b = 1, \alpha_t = 100$. We train each temporal block with the ADAM optimizer with a learning rate 10^{-3} for 1000 steps. We show in Fig. 17 their prediction profiles. With 1000 training steps, KANs have already

TABLE III. Comparing various models on the 1D Allen-Cahn equation. All MLPs including MLP_RFFs have the shape [2, 128, 128, 128, 1].

Model	ℓ_2^2 error $\downarrow$	Training wall time (s) $\downarrow$
KAN [2, 5, 5, 1] $G = 5$	3.4×10^{-3}	2801
KAN [2, 5, 5, 1] $G = 10$	3.9×10^{-3}	2831
MLP	1.5×10^{-1}	478
MLP (10 $\times$ training)	3.9×10^{-4}	4766
MLP_RFF ($s = 30$)	8.0×10^{-1}	599

learned good qualitative evolution (although with some imperfections). Training KANs for 10 000 steps probably helps, but that will take about ten hours to train, so we did not try this given the limited time during rebuttal. With 1000 training steps, MLPs do not learn the correct qualitative evolution, but adding training steps to 10 000 makes MLPs learn the evolution quite accurately. For MLPs with high-frequency random features ($s = 30$), the training curve fails to decrease, which is similar to the

observation in the Poisson case in Appendix A 2 a. Results are included in Table III.

c. Darcy flow

We use the Darcy flow to test KANs' ability to model random media (modeled as Gaussian mixtures). The equation is similar to the Poisson equation, but the permeability $a(\mathbf{x})$ can be spatially dependent [$\mathbf{x} \equiv (x, y)$]:

$$\begin{aligned} \nabla \cdot [a(\mathbf{x}) \cdot \nabla u(\mathbf{x})] &= f(\mathbf{x}), \mathbf{x} \in \Omega = [-1, 1]^2, \\ u(\mathbf{x}) &= u_t(\mathbf{x}), \mathbf{x} \in \partial\Omega, \\ a(\mathbf{x}) &= 1 + \sum_{i=1}^{N_a} \exp\left(-\frac{(x - x_{a,i}^2) + (y - y_{a,i}^2)}{2\sigma_{a,i}^2}\right), x_{a,i}, y_{a,i} \sim U[-1, 1], \sigma_{a,i} \sim U[0.1, 0.3], \\ u_t(\mathbf{x}) &= \sum_{i=1}^{N_u} \exp\left(-\frac{(x - x_{u,i}^2) + (y - y_{u,i}^2)}{2\sigma_{u,i}^2}\right), x_{u,i}, y_{u,i} \sim U[-1, 1], \sigma_{u,i} \sim U[0.1, 0.3]. \end{aligned} \quad (\text{A3})$$

Our setup is exactly the same as the Poisson equation in Appendix A 2 a, except that the ground truth solution is different, and the left differential operator is slightly different from a simple Laplacian. We visualize prediction solutions in Fig. 18 and report errors and training time in Table IV.

3. MNIST hand-written digit classification

To test KANs' scalability for high-dimensional datasets, we train KANs on MNIST. We normalize the pixel values into $[0, 1]$, and flatten the $28(\times)28$ image into a 784-dimensional vector. We train models (listed in Table V)

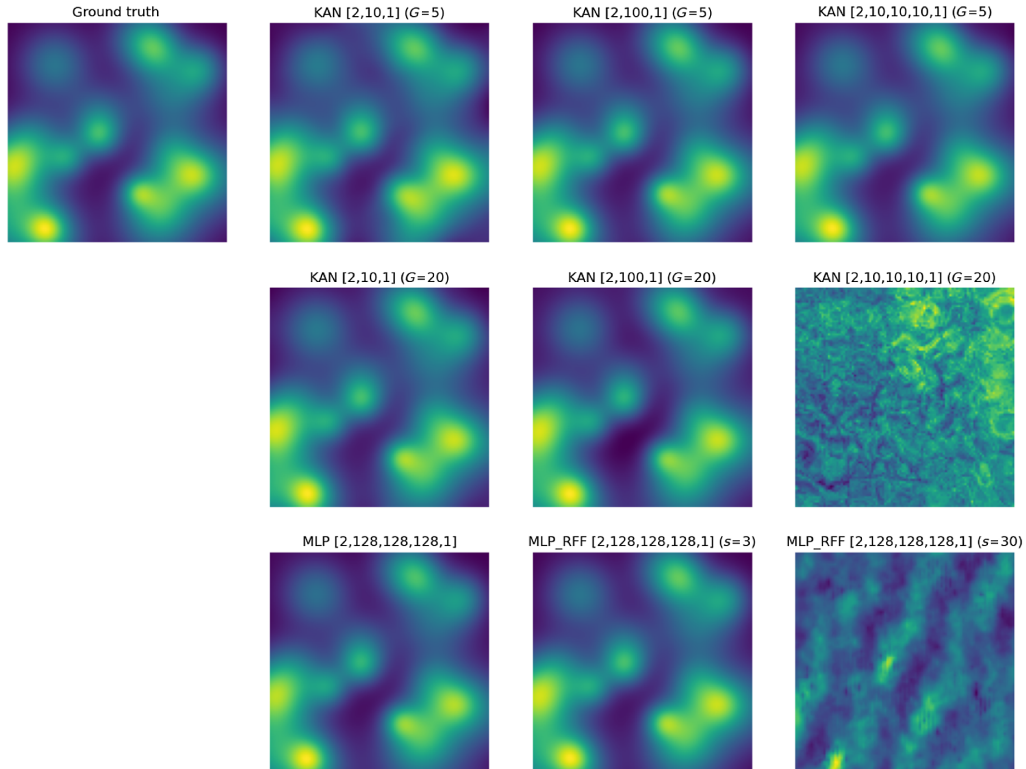


FIG. 18. Prediction of various models on Darcy flow.

TABLE IV. Comparing various models on Darcy flow. All MLPs including MLP_RFFs have the shape [2,128,128,128,1].

Model	l_2^2 error $\downarrow$	Training wall time (s) $\downarrow$
KAN [2,10,1] $G = 5$	3.9×10^{-3}	71
KAN [2,10,1] $G = 10$	1.3×10^{-3}	66
KAN [2, 10,1] $G = 20$	3.9×10^{-4}	66
KAN [2,100,1] $G = 5$	1.7×10^{-5}	81
KAN [2,100,1] $G = 10$	4.3×10^{-6}	107
KAN [2,100,1] $G = 20$	1.9×10^{-2}	136
KAN [2,10,10,10,10,1] $G = 5$	8.5×10^{-5}	123
KAN [2,10,10,10,10,1] $G = 10$	1.2	123
KAN [2,10,10,10,10,1] $G = 20$	1.3	125
MLP	3.0×10^{-5}	30
MLP (10 $\times$ training)	4.5×10^{-6}	277
MLP_RFF ($s = 3$)	5.9×10^{-6}	31
MLP_RFF ($s = 30$)	4.0×10^{-1}	31

with the ADAM optimizer (10^{-2} learning rate) for 2000 steps on the cross-entropy loss, with batch size 1024. The whole training dataset (60 000) and test dataset (10 000) are used to evaluate train, test loss, and acc. We report these metrics in Table V.

There are a few observations. (1) The shape [784,100,10] (one hidden layer of size 100) is optimal both for MLP and for KAN, which is an interesting observation. This seems to imply there is something universal across different architectures. (2) The effect of grid size: Increasing grid size decreases training loss (since it enhances fitting capability);

however, the test metrics may get worse (e.g., for [784,10]) or may get better (e.g., for [784,100,10]). Combined with (1), this seems to imply that increasing grid size is beneficial when the shape of the network is correct but might be harmful otherwise. (3) KANs and MLPs have comparable performance in terms of loss and accuracy. This is probably because the MNIST dataset is too simple. KANs consume much more training time than MLPs. Besides, the 2^k factor slowdown due to the recursive computation of order- k splines [5], grid updates are also quite expensive due to the high-dimensional inputs. We expect these slowdown factors to have straightforward solutions, and combining KANs with convolutional neural networks is a promising direction to incorporate symmetry inductive biases into architectures.

4. Remark on symbolic regression benchmarks

We have presented KANs' interpretability as an interactive tool with human users. However, as a network-based method, its strong capability (in fitting even nonsymbolic functions) makes it unfavorable for standard symbolic regression benchmarks. For example, KAN ranks second to last in GEOBENCH [114], whereas the last-ranked one EQL is also a network-based model, which has been shown to be useful at least for certain problems [97,98] despite its inability to do well on benchmarks. On the one hand, we would like to explore ways to restrict KANs' hypothesis space so that KANs can achieve good performance on symbolic regression benchmarks. On the other hand, we want to point out that KANs have good features that are not

TABLE V. Comparing KANs and MLPs on MNIST.

Model	Train loss $\downarrow$	Test loss $\downarrow$	Train acc $\uparrow$	Test acc $\uparrow$	Time (s) $\downarrow$
KAN [784, 10] $G = 3$	1.5×10^{-1}	2.8×10^{-1}	95.7%	93.0%	83.8
KAN [784, 10] $G = 5$	9.9×10^{-2}	3.3×10^{-1}	97.0%	92.4%	96.8
KAN [784, 10] $G = 10$	3.4×10^{-2}	4.4×10^{-1}	99.1%	91.7%	155.4
MLP [784, 10]	2.3×10^{-1}	2.8×10^{-1}	93.7%	92.5%	5.8
KAN [784, 10, 10] $G = 3$	8.3×10^{-2}	2.2×10^{-1}	97.5%	94.5%	106.8
KAN [784, 10, 10] $G = 5$	4.0×10^{-2}	3.1×10^{-1}	98.7%	94.2%	121.8
KAN [784, 10, 10] $G = 10$	1.8×10^{-2}	3.7×10^{-1}	99.4%	94.2%	168.0
MLP [784, 10, 10]	1.6×10^{-1}	2.3×10^{-1}	95.1%	93.7%	6.3
KAN [784, 100, 10] $G = 3$	4.0×10^{-2}	2.0×10^{-1}	99.0%	97.4%	419.3
KAN [784, 100, 10] $G = 5$	5.6×10^{-5}	9.6×10^{-2}	100.0%	98.2%	435.9
KAN [784, 100, 10] $G = 10$	3.8×10^{-5}	9.2×10^{-2}	100.0%	98.2%	531.8
MLP [784, 100, 10]	3.5×10^{-4}	9.7×10^{-2}	100.0%	97.9%	8.3
KAN [784, 100, 100, 10] $G = 3$	1.3×10^{-2}	1.8×10^{-1}	99.6%	97.6%	498.6
KAN [784, 100, 100, 10] $G = 5$	1.6×10^{-2}	1.9×10^{-1}	99.5%	97.6%	551.1
KAN [784, 100, 100, 10] $G = 10$	1.1×10^{-2}	2.0×10^{-1}	99.7%	97.5%	655.3
MLP [784, 100, 100, 10]	1.0×10^{-2}	1.5×10^{-1}	99.7%	97.7%	9.6
KAN [784, 100, 100, 100, 10] $G = 3$	1.3×10^{-2}	1.8×10^{-1}	99.6%	97.8%	631.5
KAN [784, 100, 100, 100, 10] $G = 5$	1.9×10^{-2}	1.6×10^{-1}	99.4%	97.3%	643.6
KAN [784, 100, 100, 100, 10] $G = 10$	1.5×10^{-2}	1.6×10^{-1}	99.6%	97.4%	813.3
MLP [784, 100, 100, 100, 10]	1.8×10^{-2}	1.4×10^{-1}	99.6%	97.7%	11.0

reflected by existing benchmarks. (1) Interactivity: It is relatively easier to visualize the training dynamics of KANs, which gives human users intuition on what could go wrong, hence, facilitating debugging. (2) The ability to “discover” new functions: If the ground truth formula contains a special function but is not given in the symbolic library, symbolic regression methods will fail. However, KANs can discover the need for a new function whose numerical behavior suggests maybe it may be a Bessel function.

APPENDIX B: FAILURE MODES AND HOW TO AVOID THEM

1. Existence of redundant neurons and edges with small or no sparsity penalty

In Fig. 19, we show that sparsity penalty strength λ controls the number of redundant neurons. When $\lambda = 0$, all five neurons appear to be active, with a few neurons and edges appearing to be highly similar. When $\lambda = 0.001$, only two neurons are active, and they appear almost identical (except that they differ by a minus sign). When $\lambda = 0.1$, there is only one active neuron in the hidden layer; hence, there is no redundant neuron.

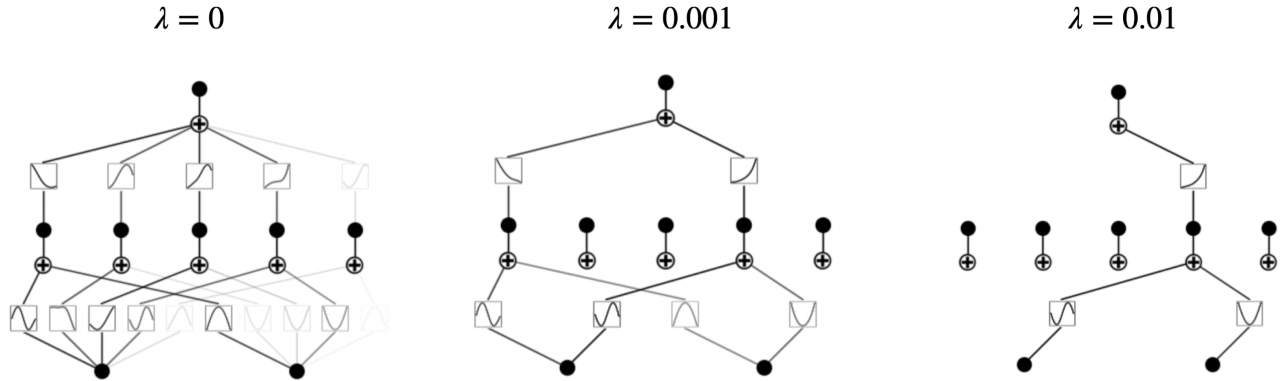


FIG. 19. Existence of redundant neurons and edges with small or no sparsity penalty. λ is the sparsity penalty strength.

2. The necessity of skip connections

In Fig. 20, we show the necessity of using skip connections, i.e., the learnable function $f(x) = b(x) + \text{spline}(x)$ with nonzero $b(x)$. By default, we choose $b(x) = \text{SiLU}(x)$. To test the necessity of such a $b(x)$, we use a simple 2D function regression task $f(x, y) = \exp[\sin(\pi x) + y^2]$. $[2, 1, 1]$ KANs ($G = 10$) are trained with the LBFGS optimizer with samples drawn from $U[-1, 1]^2$ (1000 training and 1000 test samples). We visualize KANs at step 10: KANs using sigmoid linear unit (SiLU) and linear residual connections have already learned the correct representation, while KANs without skip connections still struggle to learn the correct representations.

The intuition is that since B -splines are piecewise polynomials, they behave like order- k polynomials locally. When KANs become deeper (layer L) without skip connections, the function would become order- kL polynomials, which is quite pathological (it is known that high-order polynomials have bad numerical properties). By including the skip connections, the function can have low-order polynomial components by leveraging the skip connections.

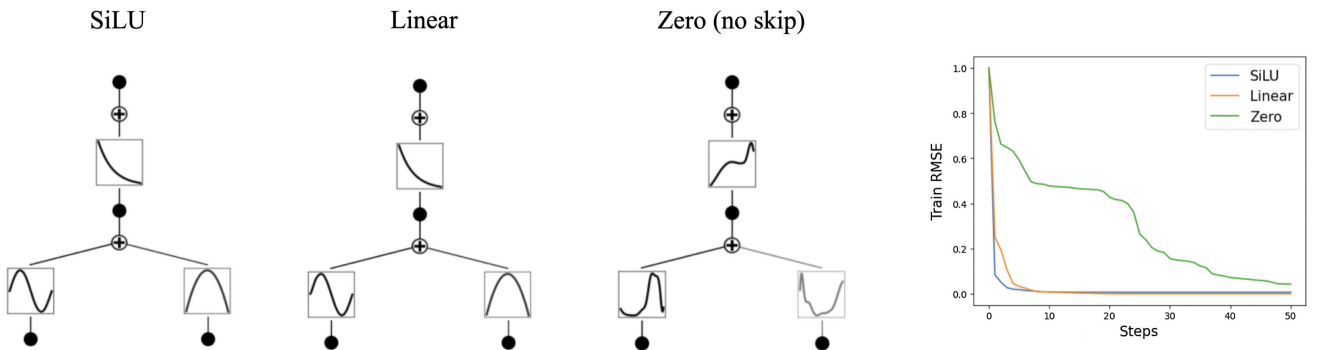


FIG. 20. Skip connections (either SiLU or linear) make training landscapes smoother, leading to faster training. The visualizations are for KANs at step 10.

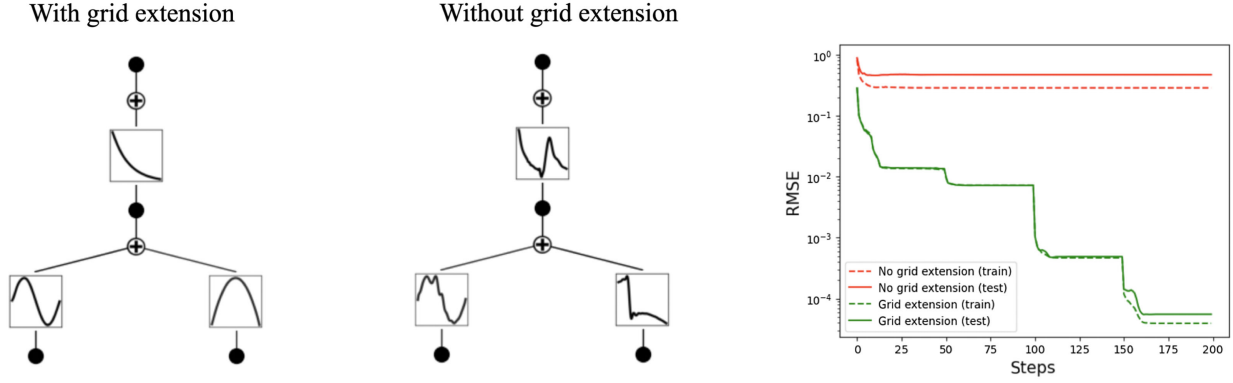


FIG. 21. Grid extension is needed to avoid bad loss landscapes when initial grid sizes are large.

3. The necessity of grid extensions

One may ask, why do we not just use the large grid size from scratch instead of using a small grid size first and then do grid extension? In Fig. 21, we show that a KAN with a large grid size can easily get stuck at local minima (probably due to a bad loss landscape). By contrast, a well-initialized KAN with a large grid size (obtained by grid extension from smaller grid sizes) does not have such a problem. With grid extension, we train the model starting from $G = 3$ for 50 steps with LBFGS, and then we do grid extension to increase G to be 5, 10, 20 (each grid is trained for another 50 steps). Without grid extension, the KAN is initialized to have $G = 20$ and is trained for 200 steps.

tions, such as $xy = [(x+y)^2 - (x-y)^2]/4$ or $xy = [(x+y)^2 - x^2 - y^2]/2$.

- [1] J. Jumper, R. Evans, A. Pritzel, T. Green, M. Figurnov, O. Ronneberger, K. Tunyasuvunakool, R. Bates, A. Žídek, A. Potapenko *et al.*, *Highly accurate protein structure prediction with alphafold*, *Nature (London)* **596**, 583 (2021).
- [2] K. Yang, A. Swope, A. Gu, R. Chalamala, P. Song, S. Yu, S. Godil, R. J. Prenger, and A. Anandkumar, *Leandoro: Theorem proving with retrieval-augmented language models*, *Adv. Neural Inf. Process. Syst.* **36**, 21573 (2024).
- [3] T. H. Trinh, Y. Wu, Q. V. Le, H. He, and T. Luong, *Solving olympiad geometry without human demonstrations*, *Nature (London)* **625**, 476 (2024).
- [4] R. Lam, A. Sanchez-Gonzalez, M. Willson, P. Wirsberger, M. Fortunato, F. Alet, S. Ravuri, T. Ewalds, Z. Eaton-Rosen, W. Hu *et al.*, *Learning skillful medium-range global weather forecasting*, *Science* **382**, 1416 (2023).
- [5] Z. Liu, Y. Wang, S. Vaidya, F. Ruehle, J. Halverson, M. Soljačić, T. Y. Hou, and M. Tegmark, *KAN: Kolmogorov-Arnold networks*, *arXiv:2404.19756*.
- [6] Y. Wang, J. W. Siegel, Z. Liu, and T. Y. Hou, *On the expressiveness and spectral bias of KANs*, *arXiv:2410.01803*.
- [7] <https://github.com/KindXiaoming/pykan>.
- [8] If x and y can be negative, one may choose a large $c > 0$ and express $xy = \exp[\log(x+c) + \log(y+c)] - c(x+y) + c^2$. Other constructions include quadratic func-
- [9] We set $k = 2$ for simplicity, but the PYKAN package allows k to be any integer $k \geq 2$. Users can even set different k values for different multiplication nodes. However, if different k values are used within the same layer, it can be challenging to parallelize these multiplications.
- [10] For subnodes belonging to the multiplication node, the subnodes inherit their scores from the multiplication node.
- [11] Other choices can be made based on the perceived importance of each output dimension, though this is less critical when outputs are typically one dimensional.
- [12] Z. Liu, E. Gan, and M. Tegmark, *Seeing is believing: Brain-inspired modular training for mechanistic interpretability*, *Entropy* **26**, 41 (2023).
- [13] Z. Liu, M. Khona, I. R. Fiete, and M. Tegmark, *Growing brains: Co-emergence of anatomical and functional modularity in recurrent neural networks*, *arXiv:2310.07711*.
- [14] S.-M. Udrescu, A. Tan, J. Feng, O. Neto, T. Wu, and M. Tegmark, *Ai FEYNMAN2.0: Pareto-optimal symbolic regression exploiting graph modularity*, *Adv. Neural Inf. Process. Syst.* **33**, 4860 (2020).
- [15] Z. Liu and M. Tegmark, *Machine learning conservation laws from trajectories*, *Phys. Rev. Lett.* **126**, 180604 (2021).
- [16] Z. Liu, V. Madhavan, and M. Tegmark, *Machine learning conservation laws from differential equations*, *Phys. Rev. E* **106**, 045307 (2022).
- [17] Z. Liu, P. O. Sturm, S. Bharadwaj, S. J. Silva, and M. Tegmark, *Interpretable conservation laws as sparse invariants*, *Phys. Rev. E* **109**, L023301 (2024).
- [18] P. Y. Lu, R. Dangovski, and M. Soljačić, *Discovering conservation laws using optimal transport and manifold learning*, *Nat. Commun.* **14**, 4744 (2023).
- [19] S. Ha and H. Jeong, *Discovering conservation laws from trajectories via machine learning*, *arXiv:2102.04008*.
- [20] S. J. Wetzel, R. G. Melko, J. Scott, M. Panju, and V. Ganesh, *Discovering symmetry invariants and conserved quantities by interpreting Siamese neural networks*, *Phys. Rev. Res.* **2**, 033499 (2020).
- [21] M. Cranmer, S. Greydanus, S. Hoyer, P. Battaglia, D. Spergel, and S. Ho, *Lagrangian neural networks*, *arXiv:2003.04630*.

- [22] C. Misner, K. Thorne, and J. Wheeler, *Gravitation* (Princeton University Press, Princeton, NJ, 2017).
- [23] Z. Liu and M. Tegmark, *Machine learning hidden symmetries*, *Phys. Rev. Lett.* **128**, 180201 (2022).
- [24] E. J. Michaud, Z. Liu, and M. Tegmark, *Precision machine learning*, *Entropy* **25**, 175 (2023).
- [25] R. Hooke, *Lectures de Potentia Restitutiva, or of Spring Explaining the Power of Springing Bodies* (John Martyn, Basel, Switzerland, 2016), 6.
- [26] W. S. Slaughter, *The Linearized Theory of Elasticity* (Springer Science & Business Media, New York, 2012).
- [27] R. W. Ogden, *Non-Linear Elastic Deformations* (Courier Corporation, New York, 1997).
- [28] R. v. Mises, *Mechanik der festen körper im plastisch-deformablen zustand*, *Nachr. Ges. Wiss. Gött. Math.-Phys. Klasse* **1913**, 582 (1913).
- [29] G. K. Batchelor, *An Introduction to Fluid Dynamics* (Cambridge University Press, Cambridge, England, 2000).
- [30] E. Sifakis and J. Barbic, *Fem simulation of 3D deformable solids: A practitioner's guide to theory, discretization and model reduction*, in *Acm Siggraph 2012 Courses* (Cambridge University Press, Cambridge, England, 2012), pp. 1–50.
- [31] B. Smith, F. D. Goes, and T. Kim, *Stable Neo-Hookean flesh simulation*, *ACM Trans. Graph.* **37**, 1 (2018).
- [32] E. M. Arruda and M. C. Boyce, *A three-dimensional constitutive model for the large stretch behavior of rubber elastic materials*, *J. Mech. Phys. Solids* **41**, 389 (1993).
- [33] A. N. Gent, *A new constitutive relation for rubber*, *Rubber Chem. Technol.* **69**, 59 (1996).
- [34] A. Sanchez-Gonzalez, J. Godwin, T. Pfaff, R. Ying, J. Leskovec, and P. Battaglia, *Learning to simulate complex physics with graph networks*, in *International Conference on Machine Learning* (PMLR, Akron, OH, 2020), pp. 8459–8468.
- [35] H. Xu, F. Sin, Y. Zhu, and J. Barbič, *Nonlinear material design using principal stretches*, *ACM Trans. Graph.* **34**, 1 (2015).
- [36] P. Ma, P. Y. Chen, B. Deng, J. B. Tenenbaum, T. Du, C. Gan, and W. Matusik, *Learning neural constitutive laws from motion observations for generalizable PDE dynamics*, in *International Conference on Machine Learning* (PMLR, Cambridge, MA, 2023), pp. 23279–23300.
- [37] P. Ma, T.-H. Wang, M. Guo, Z. Sun, J. B. Tenenbaum, D. Rus, C. Gan, and W. Matusik, *Llm and simulation as bilevel optimizers: A new paradigm to advance physical scientific discovery*, in *Forty-First International Conference on Machine Learning* (Proceedings of Machine Learning Research (PMLR), Cambridge, MA, 2024).
- [38] F. Girosi and T. Poggio, *Representation properties of networks: Kolmogorov's theorem is irrelevant*, *Neural Comput.* **1**, 465 (1989).
- [39] R. Bresson, G. Nikolentzos, G. Panagopoulos, M. Chatzianastasis, J. Pang, and M. Vazirgiannis, *Kagnns: Kolmogorov-Arnold networks meet graph learning*, *arXiv:2406.18380*.
- [40] G. De Carlo, A. Mastropietro, and A. Anagnostopoulos, *Kolmogorov-Arnold graph neural networks*, *arXiv:2406.18354*.
- [41] M. Kiamari, M. Kiamari, and B. Krishnamachari, *GKAN: Graph Kolmogorov-Arnold networks*, *arXiv:2406.06470*.
- [42] F. Zhang and X. Zhang, *GraphKAN: Enhancing feature extraction with graph Kolmogorov Arnold networks*, *arXiv:2406.13597*.
- [43] Y. Wang, J. Sun, J. Bai, C. Anitescu, M. S. Eshaghi, X. Zhuang, T. Rabczuk, and Y. Liu, *Kolmogorov Arnold informed neural network: A physics-informed deep learning framework for solving pdes based on Kolmogorov Arnold networks*, *arXiv:2406.11045*.
- [44] K. Shukla, J. D. Toscano, Z. Wang, Z. Zou, and G. E. Karniadakis, *A comprehensive and FAIR comparison between MLP and KAN representations for differential equations and operator networks*, *arXiv:2406.02917*.
- [45] D. W. Abueidda, P. Pantidis, and M. E. Mobasher, *DeepO-KAN: Deep operator network based on Kolmogorov Arnold networks for mechanics problems*, *arXiv:2405.19143*.
- [46] G. Nehma and M. Tiwari, *Leveraging KANs for enhanced deep Koopman operator discovery*, *arXiv:2406.02875*.
- [47] J. Lee, Z. Liu, X. Yu, Y. Wang, H. Jeong, M. Y. Niu, and Z. Zhang, *KANO: Kolmogorov-Arnold neural operator*, *arXiv:2509.16825*.
- [48] E. Poeta, F. Giobergia, E. Pastor, T. Cerquitelli, and E. Baralis, *A benchmarking study of Kolmogorov-Arnold networks on tabular data*, *arXiv:2406.14529*.
- [49] C. J. Vaca-Rubio, L. Blanco, R. Pereira, and M. Caus, *Kolmogorov-Arnold networks (KANs) for time series analysis*, *arXiv:2405.08790*.
- [50] R. Genet and H. Inzirillo, *TKAN: Temporal Kolmogorov-Arnold networks*, *arXiv:2405.07344*.
- [51] K. Xu, L. Chen, and S. Wang, *Kolmogorov-Arnold networks for time series: Bridging predictive power and interpretability*, *arXiv:2406.02496*.
- [52] R. Genet and H. Inzirillo, *A temporal Kolmogorov-Arnold transformer for time series forecasting*, *arXiv:2406.02486*.
- [53] M. Liu, S. Bian, B. Zhou, and P. Lukowicz, *iKAN: Global incremental learning with KAN for human activity recognition across heterogeneous datasets*, *arXiv:2406.01646*.
- [54] M. Liu, D. Geißler, D. Nshimiyimana, S. Bian, B. Zhou, and P. Lukowicz, *Initial investigation of Kolmogorov-Arnold networks (KANs) as feature extractors for IMU based human activity recognition*, *arXiv:2406.11914*.
- [55] S. Yang, L. Qin, and X. Yu, *Endowing interpretability for neural cognitive diagnosis by efficient Kolmogorov-Arnold networks*, *arXiv:2405.14399*.
- [56] L. F. Herbozo Contreras, J. Cui, L. Yu, Z. Huang, A. Nikpour, and O. Kavehei, *KAN-EEG: Towards replacing backbone-MLP for an effective seizure detection system*, *medRxiv* 2024 (2024).
- [57] A. Kundu, A. Sarkar, and A. Sadhu, *KANQAS: Kolmogorov Arnold network for quantum architecture search*, *arXiv:2406.17630*.
- [58] X. Li, Z. Feng, Y. Chen, W. Dai, Z. He, Y. Zhou, and S. Jiao, *COEFF-KANs: A paradigm to address the electrolyte field with KANs*, *arXiv:2407.20265*.
- [59] T. Ahmed and M. H. R. Sifat, *GraphKAN: Graph Kolmogorov Arnold network for small molecule-protein interaction predictions*, in *ICML'24 Workshop ML for Life and Material Science: From Theory to Industry Applications* (2024).

- [60] M. Cheon, *Kolmogorov-Arnold network for satellite image classification in remote sensing*, [arXiv:2406.00600](#).
- [61] B. Azam and N. Akhtar, *Suitability of KANs for computer vision: A preliminary investigation*, [arXiv:2406.09087](#).
- [62] C. Li, X. Liu, W. Li, C. Wang, H. Liu, and Y. Yuan, *U-KAN makes strong backbone for medical image segmentation and generation*, [arXiv:2406.02918](#).
- [63] M. Cheon, *Demonstrating the efficacy of Kolmogorov-Arnold networks in vision tasks*, [arXiv:2406.14916](#).
- [64] S. T. Seydi, *Unveiling the power of wavelets: A wavelet-based Kolmogorov-Arnold network for hyperspectral image classification*, [arXiv:2406.07869](#).
- [65] A. D. Bodner, A. S. Tepsich, J. N. Spolski, and S. Pourteau, *Convolutional Kolmogorov-Arnold networks*, [arXiv:2406.13155](#).
- [66] S. Zinage, S. Mondal, and S. Sarkar, *DKL-KAN: Scalable deep kernel learning using Kolmogorov-Arnold networks*, [arXiv:2407.21176](#).
- [67] H. Liu, J. Lei, and Z. Ren, *From complexity to clarity: Kolmogorov-Arnold networks in nuclear binding energy prediction*, [arXiv:2407.20737](#).
- [68] Y. Peng, M. He, F. Hu, Z. Mao, X. Huang, and J. Ding, *Predictive modeling of flexible EHD pumps using Kolmogorov-Arnold networks*, [arXiv:2405.07488](#).
- [69] P. Pratyush, C. Carrier, S. Pokharel, H. D. Ismail, M. Chaudhari, and D. B. KC, *CaLMPHosKAN: Prediction of general phosphorylation sites in proteins via fusion of codon aware embeddings with amino acid aware embeddings and wavelet-based Kolmogorov Arnold network*, [bioRxiv 2024 \(2024\)](#).
- [70] J. Erdmann, F. Mausolf, and J. L. Späh, *KAN we improve on hep classification tasks? Kolmogorov-Arnold networks applied to an lhc physics example*, [arXiv:2408.02743](#).
- [71] Z. Bozorgasl and H. Chen, *Wav-KAN: Wavelet Kolmogorov-Arnold networks*, [arXiv:2405.12832](#).
- [72] Z. Li, *Kolmogorov-Arnold networks are radial basis function networks*, [arXiv:2405.06721](#).
- [73] J. Xu, Z. Chen, J. Li, S. Yang, W. Wang, X. Hu, and E. C.-H. Ngai, *FourierKAN-GCF: Fourier Kolmogorov-Arnold network—an effective and efficient feature transformation for graph collaborative filtering*, [arXiv:2406.01034](#).
- [74] A. A. Howard, B. Jacob, S. H. Murphy, A. Heinlein, and P. Stinis, *Finite basis Kolmogorov-Arnold networks: Domain decomposition for data-driven and physics-informed problems*, [arXiv:2406.19662](#).
- [75] H.-T. Ta, *BSRBF-KAN: A combination of b-splines and radial basic functions in Kolmogorov-Arnold networks*, [arXiv:2406.11173](#).
- [76] A. A. Aghaei, *fKAN: Fractional Kolmogorov-Arnold networks with trainable Jacobi basis functions*, [arXiv:2406.07456](#).
- [77] S. T. Seydi, *Exploring the potential of polynomial basis functions in Kolmogorov-Arnold networks: A comparative study of different groups of polynomials*, [arXiv:2406.02583](#).
- [78] A. A. Aghaei, *rKAN: Rational Kolmogorov-Arnold networks*, [arXiv:2406.14495](#).
- [79] M. G. Altarabichi, *DropKAN: Regularizing KANs by masking post-activations*, [arXiv:2407.13044](#).
- [80] Y. Chen, Z. Zhu, S. Zhu, L. Qiu, B. Zou, F. Jia, Y. Zhu, C. Zhang, Z. Fang, F. Qin *et al.*, *SCKANsformer: Fine-grained classification of bone marrow cells via KANsformer backbone and hierarchical attention mechanisms*, [arXiv:2406.09931](#).
- [81] S. Rigas, M. Papachristou, T. Papadopoulos, F. Anagnostopoulos, and G. Alexandridis, *Adaptive training of grid-dependent physics-informed Kolmogorov-Arnold networks*, [arXiv:2407.17611](#).
- [82] S. Rigas, D. Verma, G. Alexandridis, and Y. Wang, *Initialization schemes for Kolmogorov-Arnold networks: An empirical study*, [arXiv:2509.03417](#).
- [83] E. Zeydan, C. J. Vaca-Rubio, L. Blanco, R. Pereira, M. Caus, and A. Aydeger, *F-KANs: Federated Kolmogorov-Arnold networks*, [arXiv:2407.20100](#).
- [84] T. X. H. Le, T. D. Tran, H. L. Pham, V. T. D. Le, T. H. Vu, V. T. Nguyen, Y. Nakashima *et al.*, *Exploring the limitations of Kolmogorov-Arnold networks in classification: Insights to software training and hardware implementation*, [arXiv:2407.17790](#).
- [85] H. Shen, C. Zeng, J. Wang, and Q. Wang, *Reduced effectiveness of Kolmogorov-Arnold networks on functions with noise*, [arXiv:2407.14882](#).
- [86] R. Yu, W. Yu, and X. Wang, *KAN or MLP: A fairer comparison*, [arXiv:2407.16674](#).
- [87] T. Wu and M. Tegmark, *Toward an artificial intelligence physicist for unsupervised learning*, *Phys. Rev. E* **100**, 033311 (2019).
- [88] S. L. Brunton, J. L. Proctor, and J. N. Kutz, *Discovering governing equations from data by sparse identification of nonlinear dynamical systems*, *Proc. Natl. Acad. Sci. U.S.A.* **113**, 3932 (2016).
- [89] P. Lemos, N. Jeffrey, M. Cranmer, S. Ho, and P. Battaglia, *Rediscovering orbital mechanics with machine learning*, *Mach. Learn.* **4**, 045002 (2023).
- [90] M. Cranmer, A. Sanchez Gonzalez, P. Battaglia, R. Xu, K. Cranmer, D. Spergel, and S. Ho, *Discovering symbolic models from deep learning with inductive biases*, *Adv. Neural Inf. Process. Syst.* **33**, 17429 (2020).
- [91] S. Krippendorff and M. Syvaeri, *Detecting symmetries with neural networks*, *Mach. Learn.* **2**, 015010 (2020).
- [92] J. Yang, R. Walters, N. Dehmamy, and R. Yu, *Generative adversarial symmetry discovery*, in *International Conference on Machine Learning* (PMLR, Bristol, England, 2023), pp. 39488–39508.
- [93] S. J. Wetzel, *Unsupervised learning of phase transitions: From principal component analysis to variational autoencoders*, *Phys. Rev. E* **96**, 022140 (2017).
- [94] J. Carrasquilla and R. G. Melko, *Machine learning phases of matter*, *Nat. Phys.* **13**, 431 (2017).
- [95] S. Greydanus, M. Dzamba, and J. Yosinski, *Hamiltonian neural networks*, *Adv. Neural Inf. Process. Syst.* **32** (2019).
- [96] M. Cranmer, *Interpretable machine learning for science with pysr and symbolicregression. jl*, [arXiv:2305.01582](#).
- [97] G. Martius and C. H. Lampert, *Extrapolation and learning equations*, [arXiv:1610.02995](#).
- [98] O. Dugan, R. Dangovski, A. Costa, S. Kim, P. Goyal, J. Jacobson, and M. Soljačić, *Occamnet: A fast neural model for symbolic regression at scale*, [arXiv:2007.10784](#).

- [99] M. Schmidt and H. Lipson, *Distilling free-form natural laws from experimental data*, *Science* **324**, 81 (2009).
- [100] H. Cunningham, A. Ewart, L. Riggs, R. Huben, and L. Sharkey, *Sparse autoencoders find highly interpretable features in language models*, [arXiv:2309.08600](https://arxiv.org/abs/2309.08600).
- [101] K. Meng, D. Bau, A. J. Andonian, and Y. Belinkov, *Locating and editing factual associations in GPT*, in *Advances in Neural Information Processing Systems*, edited by A. H. Oh, A. Agarwal, D. Belgrave, and K. Cho (2022).
- [102] K. R. Wang, A. Variengien, A. Conmy, B. Shlegeris, and J. Steinhardt, *Interpretability in the wild: a circuit for indirect object identification in GPT-2 small*, in *Advances in Neural Information Processing Systems (NeurIPS) Conference, New Orleans, USA* (2023).
- [103] N. Elhage, T. Hume, C. Olsson, N. Schiefer, T. Henighan, S. Kravec, Z. Hatfield-Dodds, R. Lasenby, D. Drain, C. Chen *et al.*, *Toy models of superposition*, [arXiv:2209.10652](https://arxiv.org/abs/2209.10652).
- [104] N. Nanda, L. Chan, T. Lieberum, J. Smith, and J. Steinhardt, *Progress measures for grokking via mechanistic interpretability*, in *The Eleventh International Conference on Learning Representations* (2023).
- [105] Z. Zhong, Z. Liu, M. Tegmark, and J. Andreas, *The clock and the pizza: Two stories in mechanistic explanation of neural networks*, in *International Conference on Learning Representations (ICLR) Conference, Kigali, Rwanda* (2023).
- [106] N. Elhage *et al.*, *Softmax linear units*, *Transformer Circuits Thread* (2022). <https://transformer-circuits.pub/2022/solu/index.html>.
- [107] K. Li, A. K. Hopkins, D. Bau, F. Viégas, H. Pfister, and M. Wattenberg, *Emergent world representations: Exploring a sequence model trained on a synthetic task*, in *The Eleventh International Conference on Learning Representations* (2023).
- [108] J. Engels, I. Liao, E. J. Michaud, W. Gurnee, and M. Tegmark, *Not all language model features are linear*, [arXiv:2405.14860](https://arxiv.org/abs/2405.14860).
- [109] M. Takamoto, T. Praditia, R. Leiteritz, D. MacKinlay, F. Alesiani, D. Pflüger, and M. Niepert, *Pdebench: An extensive benchmark for scientific machine learning*, *Adv. Neural Inf. Process. Syst.* **35**, 1596 (2022).
- [110] V. Sitzmann, J. Martel, A. Bergman, D. Lindell, and G. Wetzstein, *Implicit neural representations with periodic activation functions*, *Adv. Neural Inf. Process. Syst.* **33**, 7462 (2020).
- [111] L. McClenny and U. Braga-Neto, *Self-adaptive physics-informed neural networks using a soft attention mechanism*, [arXiv:2009.04544](https://arxiv.org/abs/2009.04544).
- [112] S. Wang, S. Sankaran, and P. Perdikaris, *Respecting causality is all you need for training physics-informed neural networks*, [arXiv:2203.07404](https://arxiv.org/abs/2203.07404).
- [113] S. Wang, B. Li, Y. Chen, and P. Perdikaris, *Piratenets: Physics-informed deep learning with residual adaptive networks*, [arXiv:2402.00326](https://arxiv.org/abs/2402.00326).
- [114] Y. Xu, K. Ruan, M. Liu, and H. Sun, *Geobench: A new benchmark on symbolic regression with geometric expressions* (2025), <https://openreview.net/forum?id=TqzNI4v9DT>.